```cpp
/*
  0. 数组空间是否足够? 数组空间是否足够? 数组空间是否足够? 数据范围开int还是long long?

  1. 多想一点, 少错一点!
  是否有你的算法的反例?

  2. 注意边界条件!
  n = 1考虑了吗? p = 0考虑了吗? 是否对有关-1的东西有特判?

  3. 不要犯愚蠢的错误!
  时间复杂度有没有写假? 空间足够吗? 精度是否正确?
*/

#include <bits/extc++.h>
using ll = long long;
using ull = unsigned long long;

inline void solve() {

}

int main() {
        std::ios_base::sync_with_stdio(false);
        std::cin.tie(nullptr);
        int n = 1;
        // std::cin >> n;
        for(int i = 1; i < n; ++i) {
                solve();
        }
        return 0;
}
```

板子

目录

让代码跑起来

编译运行

```
g++ -std=c++23 a.cpp -o a.out
./a.out
```

对拍器

```
# 请根据实际的C++版本修改-std版本
g++ generator.cpp -o generator.out -std=c++20 -O2 || exit 1
g++ solution.cpp -o solution.out -std=c++20 -O2 || exit 1
g++ brute_force.cpp -o brute_force.out -std=c++20 -O2 || exit 1
mkdir -p testcases
count=1
while true; do
    echo "Running test $count"
    ./generator.out > testcases/input.txt
    timeout 2s ./solution.out < testcases/input.txt > testcases/output_solution.txt
    exit_code=$?
    if [ $exit_code -eq 124 ]; then
        echo -e "\033[31mTime Limit Exceeded on solution\033[0m"
        break
    elif [ $exit_code -eq 139 ]; then
        echo -e "\033[31mRuntime Error (Segmentation fault) on solution\033[0m"
        cat testcases/input.txt
        break
    elif [ $exit_code -eq 134 ]; then
        echo -e "\033[31mRuntime Error (Aborted) on solution\033[0m"
        cat testcases/input.txt
        break
    elif [ $exit_code -eq 136 ]; then
        echo -e "\033[31mRuntime Error (Floating point exception) on solution\033[0m"
        cat testcases/input.txt
        break
    fi
    timeout 2s ./brute_force.out < testcases/input.txt > testcases/output_brute_force.txt
    if [ $exit_code -eq 124 ]; then
        echo -e "\033[31mTime Limit Exceeded on brute force\033[0m"
        break
    fi
    if ! diff -wB testcases/output_solution.txt testcases/output_brute_force.txt > /dev/null; then
        echo -e "\033[31mWrong Answer on test case $count\033[0m"
        echo "Input:"
        cat testcases/input.txt
        echo -e "\nSolution output:"
        cat testcases/output_solution.txt
        echo -e "\nBrute force output:"
        cat testcases/output_brute_force.txt
        break
    fi
    echo -e "\033[32mAccepted\033[0m"
    ((count++))
done
```

Debug宏

```
#ifdef ONLINE_JUDGE
#define debug(...) (void(0))
#else

string trim(string s) {
    int l = 0, r = s.size() - 1;
    while(l <= r && isspace(s[l])) ++l;
    while(r >= l && isspace(s[r])) --r;
    return s.substr(l, r - l + 1);
}

vector<string> split_args(const string &s) {
    vector<string> ret;
    string cur;
    int dep = 0;
    for(char ch : s) {
        if(ch == '(' || ch == '[') {
            ++dep;
            cur += ch;
        } else if(ch == ')' || ch == ']') {
            --dep;
            cur += ch;
        } else if(dep == 0 && ch == ',') {
            ret.push_back(trim(cur));
            cur = "";
        } else {
            cur += ch;
        }
    }
    if(!(trim(cur).empty())) ret.push_back(trim(cur));
    return ret;
}

template<class Tuple, size_t ... I> void print_tuple
        (const vector<string> &names, const Tuple &t, index_sequence<I...>) {
    using expr = int[];
    bool first = true;
    (void)expr{0, ((void)(
        (first ? first = false : (bool)(cerr << ", ")),
        cerr << names[I] << " = " << get<I>(t)
    ), 0)...};
}

template<class ... Args> void debug_helper(const string &s, Args &&... args) {
    auto names = split_args(s);
    auto tp = forward_as_tuple(forward<Args>(args)...);
    print_tuple(names, tp, make_index_sequence<sizeof...(Args)>{});
    cerr << endl;
}

#define debug(...) debug_helper(#__VA_ARGS__, __VA_ARGS__)
#endif
```

数据结构

线段树

```
template<class Info> concept SegInfo = requires(Info a, Info b) {
        { a + b } -> same_as<Info>;
        { a.update(b) } -> same_as<void>;
};

template<SegInfo Info> struct SegTree {
public:
        SegTree() : n(0) {}
        explicit SegTree(int sz) : n(sz), info(sz * 4, Info()) {}
        explicit SegTree(const vector<Info> &v) : n(v.size()), info(v.size() * 4) {
                _build(v, 0, 0, n);
        }
        void assign(int sz) {
                n = sz;
                info.assign(n * 4, Info());
        }
        void assign(const vector<Info> &v) {
                n = v.size();
                info.assign(n * 4, Info());
                _build(v, 0, 0, n);
        }
        Info query(int l, int r) const {
                if(l == r) return Info();
                return _query(l, r, 0, 0, n);
        }
        void update(int x, const Info &v) {
                _update(x, v, 0, 0, n);
        }
private:
        int n;
        vector<Info> info;
        void _build(const vector<Info> &v, int u, int rl, int rr) {
                if(rl == rr - 1) {
                        info[u] = v[rl];
                        return;
                }
                int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
                _build(v, ls, rl, mid);
                _build(v, rs, mid, rr);
                info[u] = info[ls] + info[rs];
        }
        Info _query(int ql, int qr, int u, int rl, int rr) const {
                if(ql <= rl && qr >= rr) return info[u];
                int mid = (rl + rr) >> 1, ls = u * 2 + 1, rs = u * 2 + 2;
                Info res{};
                if(ql < mid) res = res + _query(ql, qr, ls, rl, mid);
                if(qr > mid) res = res + _query(ql, qr, rs, mid, rr);
                return res;
        }
        void _update(int x, const Info &v, int u, int rl, int rr) {
                if(rl == rr - 1) {
                        v.update(info[u]);
```

```
                        return;
                }
                int mid = (rl + rr) >> 1, ls = u * 2 + 1, rs = u * 2 + 2;
                if(x < mid) _update(x, v, ls, rl, mid);
                else _update(x, v, rs, mid, rr);
                info[u] = info[ls] + info[rs];
        }
};

template<class Info, class Tag> concept LazySegInfoTag = requires(Info a, Info b, Tag c, Tag d, int l, int r) {
        { a + b } -> same_as<Info>;
        { a.update(b, l, r) } -> same_as<void>;
        { a.update(c) } -> same_as<void>;
        { c.apply(a, l, r) } -> same_as<void>;
        { c.apply(d) } -> same_as<void>;
        { c.clear() } -> same_as<void>;
        { c.is_null() } -> same_as<bool>;
};
// 懒标记线段树正在施工中，现在请不要使用
template<class Info, class Tag> requires LazySegInfoTag<Info, Tag> struct LazySegTree {
        LazySegTree() : LazySegTree(0) {}
        explicit LazySegTree(int n) : n(n), info(4 * n, Info{}), tag(4 * n, Tag{}) {}
        explicit LazySegTree(const vector<Info> &v) : n(v.size()), info(4 * n), tag(4 * n) {
                _build(v, 0, 0, n);
        }
        void assign(int _n) {
                n = _n;
                info.assign(4 * n, Info{});
                tag.assign(4 * n, Tag{});
        }
        void assign(const vector<Info> &v) {
                n = v.size();
                info.assign(4 * n, Info{});
                tag.assign(4 * n, Tag{});
                _build(v, 0, 0, n);
        }
        Info query(int l, int r) {
                return _query(l, r, 0, 0, n);
        }
        void update(int l, int r, const Info &dv) {
                _update(l, r, dv, 0, 0, n);
        }
private:
        int n;
        vector<Info> info;
        vector<Tag> tag;
        void _build(const vector<Info> &v, int u, int rl, int rr) {
                if(rl + 1 == rr) {
                        info[u] = v[rl];
                        return;
                }
                int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
                _build(v, ls, rl, mid);
                _build(v, rs, mid, rr);
                info[u] = info[ls] + info[rs];
        }
        void _pushdown(int u, int rl, int rr) {
```

```
                if(tag[u].is_null()) return;
                int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
                tag[u].apply(info[ls], rl, mid);
                tag[u].apply(info[rs], mid, rr);
                tag[u].apply(tag[ls]);
                tag[u].apply(tag[rs]);
                tag[u].clear();
        }
        Info _query(int ql, int qr, int u, int rl, int rr) {
                if(ql <= rl && rr <= qr) return info[u];
                _pushdown(u, rl, rr);
                int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
                Info ret{};
                if(ql < mid) {
                        ret = ret + _query(ql, qr, ls, rl, mid);
                }
                if(qr > mid) {
                        ret = ret + _query(ql, qr, rs, mid, rr);
                }
                return ret;
        }
        void _update(int ul, int ur, const Info &dv, int u, int rl, int rr) {
                if(ul <= rl && rr <= ur) {
                        dv.update(info[u], rl, rr);
                        dv.update(tag[u]);
                        return;
                }
                _pushdown(u, rl, rr);
                int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
                if(ul < mid) {
                        _update(ul, ur, dv, ls, rl, mid);
                }
                if(ur > mid) {
                        _update(ul, ur, dv, rs, mid, rr);
                }
                info[u] = info[ls] + info[rs];
        }
};
```

权值线段树

```
// value segment tree
struct VST {
	explicit VST(int maxn) : sum((maxn + 1) * 4), n(maxn + 1) {}
	VST() : n(0) {}
	void assign(int maxn) {
		sum.assign((maxn + 1) * 4, 0);
		n = maxn + 1;
	}
	void insert(int x) {
		_update(x, 1, 0, 0, n);
	}
	void erase(int x) {
		_update(x, -1, 0, 0, n);
	}
	int qpre(int x) const {
		int rk = qrv(x);
		return qvr(rk - 1);
	}
	int qsuc(int x) const {
		int rk = qrv(x + 1);
		return qvr(rk);
	}
	int qrv(int x) const {
		return _query(0, x, 0, 0, n);
	}
	int qvr(int k) const {
		return _qvr(k, 0, 0, n);
	}
	int size() const {
		return sum[0];
	}
	int qcnt(int x) const {
		return _query(x, x + 1, 0, 0, n);
	}
	int qrange_cnt(int l, int r) const {
		return _query(l, r, 0, 0, n);
	}
private:
	vector<int> sum;
	int n;
	void _update(int x, int dv, int rt, int rl, int rr) {
		if(rr - rl == 1) {
			sum[rt] += dv;
			return;
		}
		int mid = (rl + rr) >> 1, lson = rt * 2 + 1, rson = rt * 2 + 2;
		if(x < mid) {
			_update(x, dv, lson, rl, mid);
		} else {
			_update(x, dv, rson, mid, rr);
		}
		sum[rt] = sum[lson] + sum[rson];
	}
	ll _query(int ql, int qr, int rt, int rl, int rr) const {
		if(ql <= rl && qr >= rr)
```

```
                return sum[rt];
        int mid = (rl + rr) >> 1, lson = rt * 2 + 1, rson = rt * 2 + 2;
        ll ans = 0;
        if(ql < mid) {
                ans += _query(ql, qr, lson, rl, mid);
        }
        if(qr > mid) {
                ans += _query(ql, qr, rson, mid, rr);
        }
        return ans;
    }
    int _qvr(int k, int rt, int rl, int rr) const {
        if(rr - rl == 1)
                return rl;
        int mid = (rl + rr) >> 1, lson = rt * 2 + 1, rson = rt * 2 + 2;
        if(k < sum[lson]) {
                return _qvr(k, lson, rl, mid);
        } else {
                return _qvr(k - sum[lson], rson, mid, rr);
        }
    }
};
```

主席树

```
// persistent segment tree
struct PST {
        const int n;
        explicit PST(int n_) : n(n_) {
                nodes.reserve(2 * n * (int)log2(n));
                vers.reserve(n + 1);
                vers.push_back(_build(0, n - 1));
        }
        void update(int k) {
                vers.push_back(_update(k, vers.back(), 0, n - 1));
        }
        // 这里为解决静态区间第k小问题设计，传入k, l, r
        int query(int k, int ver1, int ver2) const {
                return _query(k, vers[ver1 - 1], vers[ver2], 0, n - 1);
        }
private:
        struct Node {
                int l, r;
                int cnt;
        };
        vector<Node> nodes;
        vector<int> vers;
        int _build(int l, int r) {
                int ret = nodes.size();
                if(l == r) {
                        nodes.emplace_back(-1, -1, 0);
                        return ret;
                }
                nodes.emplace_back(-1, -1, 0);
                int mid = (l + r) >> 1;
                nodes[ret].l = _build(l, mid);
                nodes[ret].r = _build(mid + 1, r);
                return ret;
        }
        int _update(int k, int root, int l, int r) {
                int ret = nodes.size();
                nodes.emplace_back(nodes[root]);
                nodes[ret].cnt++;
                if(l == r) {
                        return ret;
                }
                int mid = (l + r) >> 1;
                if(k <= mid) {
                        nodes[ret].l = _update(k, nodes[root].l, l, mid);
                } else {
                        nodes[ret].r = _update(k, nodes[root].r, mid + 1, r);
                }
                return ret;
        }
        int _query(int k, int root1, int root2, int l, int r) const {
                if(l == r) return l;
                int mid = (l + r) >> 1;
                int cntl = nodes[nodes[root2].l].cnt - nodes[nodes[root1].l].cnt;
                if(k <= cntl) {
                        return _query(k, nodes[root1].l, nodes[root2].l, l, mid);
```

```
                } else {
                        return _query(k - cntl, nodes[root1].r, nodes[root2].r, mid + 1, r);
                }
        }
};
```

树状数组（单点、区间）

```
// 单点
struct Fenwick {
        explicit Fenwick(int n) : n(n), nums(n + 1, 0) {}
        ll query(int x) const {
                ll ans = 0;
                for(; x; x -= lbit(x)) {
                        ans += nums[x];
                }
                return ans;
        }
        void update(int x, ll v) {
                for(; x <= n; x += lbit(x)) {
                        nums[x] += v;
                }
        }
private:
        vector<ll> nums;
        int n;
        static int lbit(int x) {
                return x & -x;
        }
};
// 区间
struct RangeFenwick {
        const int n;
        RangeFenwick(int n)
                : n(n), f1(n), f2(n) {}
        void update(int l, int r, ll v) {
                _update(l, v);
                _update(r + 1, -v);
        }
        ll query(int l, int r) const {
                return _query(r) - _query(l - 1);
        }
private:
        Fenwick f1, f2;
        void _update(int x, ll v) {
                f1.update(x, v);
                f2.update(x, v * (x - 1));
        }
        ll _query(int x) const {
                return f1.query(x) * x - f2.query(x);
        }
};
```

差分数组

使用差分数组区间加等差数列

```
// 二次差分
vector<ll> diff(n + 3, 0);
// [1,l]第i项加i
diff[1] += val;
diff[l + 1] -= val;
// [r+1,l+r]第i项减i-r
diff[r + 1] -= val;
diff[l + r + 1] += val;
// 还原
for(int _ = 0; _ < 2; ++_) {
        for(int i = 1; i < n + 3; ++i) {
                diff[i] += diff[i - 1];
        }
}
```

树状数组在线处理

```
struct ArithmeticFenwick {
        explicit ArithmeticFenwick(int n)
                : f1(n + 2), f2(n + 2) {}
        // a*idx+b
        void update(int l, int r, ll a, ll b) {
                f1.update(l, b);
                f1.update(r + 1, -b - a * (r - l + 1));
                f2.update(l, a);
                f2.update(r + 1, -a);
        }
        ll query(ll idx) const {
                return f1.query(idx) + idx * f2.query(idx);
        }
private:
        Fenwick f1, f2;
};
```

分块

```
struct Block {
        Block() : l(-1), r(-1) {}
        // 注意左闭右开
        Block(int l, int r, const vector<ll> &nums_)
                : nums(r - l), lazy(0), sum(0), l(l), r(r) {
                for(int i = l; i < r; ++i) {
                        nums[i - l] = nums_[i];
                        sum += nums[i - l];
                }
        }
        // 注意左闭右开
        void update(int ul, int ur, ll dval) {
                if(ul > r || ur < l) {
                        return;
                }
                if(ul <= l && ur >= r) {
                        lazy += dval;
                        sum += dval * (r - l);
                        return;
                }
                for(int i = max(ul, l); i < min(ur, r); ++i) {
                        nums[i - l] += dval;
                }
                sum += dval * (min(ur, r) - max(ul, l));
        }
        // 注意左闭右开
        ll query(int ql, int qr) const {
                if(ql >= r || qr <= l) {
                        return 0;
                }
                if(ql <= l && qr >= r) {
                        return sum;
                }
                ll ans = lazy * (min(qr, r) - max(ql, l));
                for(int i = max(ql, l); i < min(qr, r); ++i) {
                        ans += nums[i - l];
                }
                return ans;
        }
private:
        vector<ll> nums;
        ll lazy;
        ll sum;
        int l, r;
};
struct BlockedArray {
        explicit BlockedArray(const vector<ll> &nums) {
                int r = nums.size();
                int len = sqrt(r + 0.5);
                int size = (r + len - 1) / len;
                blocks.reserve(size);
                for(int i = 0; i < size; ++i) {
                        int ll = i * len, rr = min((i + 1) * len, r);
                        blocks.emplace_back(ll, rr, nums);
                }
```

```
        }
        void update(int l, int r, ll d) {
                for(auto &blk : blocks) {
                        blk.update(l, r, d);
                }
        }
        ll query(int l, int r) const {
                ll ans = 0;
                for(const auto &blk : blocks) {
                        ans += blk.query(l, r);
                }
                return ans;
        }
        vector<Block> blocks;
};
```

并查集

```
// 按秩合并
struct DSU {
	explicit DSU(int n) : fa(n), rk(n, 1) {
		iota(fa.begin(), fa.end(), 0);
	}
	int find(int x) {
		if(fa[x] != x) {
			fa[x] = find(fa[x]);
		}
		return fa[x];
	}
	bool is_connected(int x, int y) {
		return find(x) == find(y);
	}
	void connect(int x, int y) {
		x = find(x);
		y = find(y);
		if(x == y) return;
		if(rk[x] < rk[y]) {
			swap(x, y);
		}
		if(rk[x] == rk[y]) {
			rk[x]++;
		}
		fa[y] = x;
	}
private:
	vector<int> fa, rk;
};
// 随机合并
struct DSU_Random {
	explicit DSU_Random(int n) : fa(n) {
		iota(fa.begin(), fa.end(), 0);
	}
	int find(int x) {
		if(fa[x] != x) {
			fa[x] = find(fa[x]);
		}
		return fa[x];
	}
	bool is_connected(int x, int y) {
		return find(x) == find(y);
	}
	void connect(int x, int y) {
		int fx = find(x), fy = find(y);
		if(rnd() % 2) {
			fa[fx] = fy;
		} else {
			fa[fy] = fx;
		}
	}
private:
	static inline mt19937 rnd{ (unsigned int)chrono::steady_clock::now()
		.time_since_epoch().count() };
```

```
        vector<int> fa;
};
```

无旋Treap

```
constexpr int inf = 0x3f3f3f3f;
struct Treap {
        Treap() : rt(nullptr) {}
        ~Treap() {
                if(rt) {
                        _dtor(rt);
                }
        }
        void insert(int v) {
                auto tmp = _sval(rt, v);
                auto l = _sval(tmp.first, v - 1);
                Node *node = l.second;
                if(!l.second) {
                        node = new Node(v);
                } else {
                        l.second->cnt++;
                        l.second->usize();
                }
                Node *lc = _merge(l.first, node);
                rt = _merge(lc, tmp.second);
        }
        void erase(int v) {
                auto tmp = _sval(rt, v);
                auto l = _sval(tmp.first, v - 1);
                if(l.second->cnt > 1) {
                        l.second->cnt--;
                        l.second->usize();
                        l.first = _merge(l.first, l.second);
                } else {
                        if(tmp.first == l.second) {
                                tmp.first = nullptr;
                        }
                        delete l.second;
                        l.second = nullptr;
                }
                rt = _merge(l.first, tmp.second);
        }
        // 注意0-based
        int qrv(int v) const {
                return _qrv(rt, v);
        }
        // 注意0-based
        int qvr(int r) const {
                return _qvr(rt, r);
        }
        // 注意是小于的，不是大于等于的
        int lower_bound(int v) const {
                auto tmp = _sval(rt, v - 1);
                int ret = -inf;
                if(tmp.first) {
                        ret = _qvr(tmp.first, tmp.first->size - 1);
                }
                rt = _merge(tmp.first, tmp.second);
                return ret;
        }
```

```
	int upper_bound(int v) const {
		auto tmp = _sval(rt, v);
		int ret = -inf;
		if(tmp.second) {
			ret = _qvr(tmp.second, 0);
		}
		rt = _merge(tmp.first, tmp.second);
		return ret;
	}
private:
	struct Node {
		int val;
		int cnt;
		int size;
		int prio;
		Node *left, *right;
		static inline random_device rnd{};
		Node(int v)
			: val(v), cnt(1), size(1), prio(rnd()), left(nullptr), right(nullptr) {}
		void usize() {
			size = cnt;
			if(left) {
				size += left->size;
			}
			if(right) {
				size += right->size;
			}
		}
	};
	mutable Node *rt;
	static void _dtor(Node *ptr) {
		if(ptr->left) {
			_dtor(ptr->left);
			ptr->left = nullptr;
		}
		if(ptr->right) {
			_dtor(ptr->right);
			ptr->right = nullptr;
		}
		delete ptr;
	}
	static pair<Node *, Node *> _sval(Node *const ptr, int key) {
		if(!ptr) {
			return { nullptr, nullptr };
		}
		if(ptr->val <= key) {
			auto tmp = _sval(ptr->right, key);
			ptr->right = tmp.first;
			ptr->usize();
			return { ptr, tmp.second };
		} else {
			auto tmp = _sval(ptr->left, key);
			ptr->left = tmp.second;
			ptr->usize();
			return { tmp.first, ptr };
		}
	}
```

```
	static tuple<Node *, Node *, Node *> _srnk(Node *const ptr, int rnk) {
		if(!ptr) return { nullptr, nullptr, nullptr };
		int lsize = (ptr->left ? ptr->left->size : 0);
		if(rnk < lsize) {
			auto [lptr, mptr, rptr] = _srnk(ptr->left, rnk);
			ptr->left = rptr;
			ptr->usize();
			return { lptr, mptr, ptr };
		} else if(rnk < lsize + ptr->cnt) {
			Node *lptr = ptr->left;
			Node *rptr = ptr->right;
			ptr->left = ptr->right = nullptr;
			return { lptr, ptr, rptr };
		} else {
			auto [lptr, mptr, rptr] = _srnk(ptr->right, rnk - lsize - ptr->cnt);
			ptr->right = lptr;
			ptr->usize();
			return { ptr, mptr, rptr };
		}
	}
	static Node *_merge(Node *const u, Node *const v) {
		if(!u) return v;
		if(!v) return u;
		if(u->prio < v->prio) {
			u->right = _merge(u->right, v);
			u->usize();
			return u;
		} else {
			v->left = _merge(u, v->left);
			v->usize();
			return v;
		}
	}
	int _qrv(Node *const ptr, int val) const {
		auto [l, r] = _sval(ptr, val - 1);
		int ret = (l ? l->size : 0);
		rt = _merge(l, r);
		return ret;
	}
	int _qvr(Node *const ptr, int r) const {
		auto [lptr, mptr, rptr] = _srnk(ptr, r);
		int ret = mptr->val;
		rt = _merge(_merge(lptr, mptr), rptr);
		return ret;
	}
};
```

树的倍增算法

```
struct BinaryLift {
        explicit BinaryLift(int n) : tree(n), anc(n, vector<int>(LOG, -1)), depth(n) {}
        void addedge(int u, int v) {
                tree[u].push_back(v);
                tree[v].push_back(u);
        }
        void build(int root) {
                dfs(root, -1);
        }
        int lca(int u, int v) const {
                if(depth[u] < depth[v]) swap(u, v);
                for(int k = LOG - 1; k >= 0; --k) {
                        if(anc[u][k] != -1) {
                                if(depth[anc[u][k]] >= depth[v]) {
                                        u = anc[u][k];
                                }
                        }
                }
                if(u == v) return u;
                for(int k = LOG - 1; k >= 0; --k) {
                        if(anc[u][k] != anc[v][k]) {
                                u = anc[u][k];
                                v = anc[v][k];
                        }
                }
                return anc[u][0];
        }
        int kth_ancestor(int x, int k) const {
                for(int i = 0; i < LOG; ++i) {
                        if((k >> i) & 1) {
                                x = anc[x][i];
                                if(x == -1) return -1;
                        }
                }
                return x;
        }
        vector<vector<int>> tree;
private:
        static constexpr int LOG = 20;
        vector<vector<int>> anc;
        vector<int> depth;
        void dfs(int root, int fa) {
                depth[root] = fa != -1 ? depth[fa] + 1 : 0;
                anc[root][0] = fa;
                for(int k = 1; k < LOG; ++k) {
                        if(anc[root][k - 1] == -1) {
                                anc[root][k] = -1;
                        } else {
                                anc[root][k] = anc[anc[root][k - 1]][k - 1];
                        }
                }
                for(int v : tree[root]) {
                        if(v != fa) {
                                dfs(v, root);
                        }
```

```
                }
        }
};
```

树链剖分

```
struct HLDInfo {
	ll val;
	explicit HLDInfo(ll v) : val(v) {}
	HLDInfo() : val(0) {}
	void update(HLDInfo &dst) const {
		dst.val = val;
	}
	HLDInfo operator+(const HLDInfo &x) const {
		return HLDInfo(val + x.val);
	}
};

struct Node {
	ll w;
	vector<int> e;
};
struct HLD {
	explicit HLD(const vector<Node> &g, int r = 0)
		: nodes(g.size()), nfd(g.size()) {
		dfs1(g, r, -1);
		int now_index = 0;
		dfs2(g, r, -1, now_index, r);
		vector<HLDInfo> nums(g.size());
		for(int i = 0; i < g.size(); ++i) {
			nums[nodes[i].dfn] = HLDInfo{g[i].w};
		}
		seg.assign(nums);
	}
	int lca(int u, int v) const {
		while(nodes[u].toc != nodes[v].toc) {
			if(nodes[nodes[u].toc].dep < nodes[nodes[v].toc].dep) {
				v = nodes[nodes[v].toc].fa;
			} else {
				u = nodes[nodes[u].toc].fa;
			}
		}
		return nodes[u].dep < nodes[v].dep ? u : v;
	}
	void modify(int x, ll v) {
		seg.update(nodes[x].dfn, HLDInfo{v});
	}
	ll query_sum(int u, int v) const {
		int lca_node = lca(u, v);
		auto query_to_lca = [&](int point) -> ll {
			ll ans = 0;
			for(; nodes[point].toc != nodes[lca_node].toc;
				point = nodes[nodes[point].toc].fa) {
				int pos1 = nodes[point].dfn;
				int pos2 = nodes[nodes[point].toc].dfn;
				ans += seg.query(pos2, pos1 + 1).val;
			}
			int pos1 = nodes[lca_node].dfn;
			int pos2 = nodes[point].dfn;
			ans += seg.query(pos1 + 1, pos2 + 1).val;
			return ans;
```

```
                };
                ll ans = query_to_lca(u);
                ans += query_to_lca(v);
                int lca_pos = nodes[lca_node].dfn;
                ans += seg.query(lca_pos, lca_pos + 1).val;
                return ans;
        }
private:
        struct _Node {
                int dep, fa, toc, dfn, sz, hs;
        };
        void dfs1(const vector<Node> &g, int x, int fa) {
                if(fa == -1) {
                        nodes[x].dep = 0;
                } else {
                        nodes[x].dep = nodes[fa].dep + 1;
                }
                nodes[x].fa = fa;
                nodes[x].sz = 1;
                nodes[x].hs = -1;
                for(int i = 0; i < g[x].e.size(); ++i) {
                        int next = g[x].e[i];
                        if(next == fa) continue;
                        dfs1(g, next, x);
                        nodes[x].sz += nodes[next].sz;
                        if(nodes[x].hs == -1 ||
                                nodes[nodes[x].hs].sz < nodes[next].sz) {
                                nodes[x].hs = next;
                        }
                }
        }
        void dfs2(const vector<Node> &g, int x, int fa, int &dfn, int toc) {
                nodes[x].dfn = dfn++;
                nodes[x].toc = toc;
                if(nodes[x].hs != -1) {
                        dfs2(g, nodes[x].hs, x, dfn, toc);
                        for(int i = 0; i < g[x].e.size(); ++i) {
                                int next = g[x].e[i];
                                if(next != nodes[x].hs && next != fa) {
                                        dfs2(g, next, x, dfn, next);
                                }
                        }
                }
        }
        vector<_Node> nodes;
        vector<int> nfd;
        SegTree<HLDInfo> seg;
};
```

动态BitSet

```
struct DynamicBitSet {
    explicit DynamicBitSet(int n = 0) : nums((n + 63) >> 6, 0), sz(n) {}
    void resize(int n) {
        nums.resize(n);
        sz = n;
    }
    bool getbit(int x) const {
        int u = x >> 6;
        int v = x - (u << 6);
        return ((nums[u] >> v) & 1) == 1;
    }
    void setbit(int x, bool val) {
        int u = x >> 6;
        int v = x - (u << 6);
        if(!val) {
            ull d = (ull)-1;
            d ^= (1ull << v);
            nums[u] &= d;
        } else {
            nums[u] |= (1ull << v);
        }
    }
    DynamicBitSet &operator&=(const DynamicBitSet &other) {
        for(int i = 0; i < nums.size(); ++i) {
            nums[i] &= other.nums[i];
        }
        return *this;
    }
    DynamicBitSet operator&(const DynamicBitSet &other) const {
        DynamicBitSet ret = *this;
        ret &= other;
        return ret;
    }
    DynamicBitSet &operator|=(const DynamicBitSet &other) {
        for(int i = 0; i < nums.size(); ++i) {
            nums[i] |= other.nums[i];
        }
        return *this;
    }
    DynamicBitSet operator|(const DynamicBitSet &other) const {
        DynamicBitSet ret = *this;
        ret |= other;
        return ret;
    }
    DynamicBitSet &operator^=(const DynamicBitSet &other) {
        for(int i = 0; i < nums.size(); ++i) {
            nums[i] ^= other.nums[i];
        }
        return *this;
    }
    DynamicBitSet operator^(const DynamicBitSet &other) const {
        DynamicBitSet ret = *this;
        ret ^= other;
        return ret;
    }
```

```
    bool allzero() const {
        for(ull i : nums) {
            if(i != 0ull) return false;
        }
        return true;
    }
    int size() const {return sz;}
private:
    vector<ull> nums;
    int sz;
};
```

ST表

```
template<class T> struct Sparse {
        using func_type = function<T(T, T)>;
        Sparse(const vector<T> &vec, func_type fn) : func(fn) {
                int n = vec.size();
                _log.resize(n + 1);
                _log[0] = -1;
                for(int i = 1; i <= n; ++i) {
                        _log[i] = _log[i >> 1] + 1;
                }
                int k = _log[n] + 1;
                table.resize(n, vector<T>(k));
                for(int i = 0; i < n; ++i) {
                        table[i][0] = vec[i];
                }
                for(int j = 1; j < k; ++j) {
                        int step = 1 << (j - 1);
                        for(int i = 0; i + step < n; ++i) {
                                table[i][j] = func(table[i][j - 1], table[i + step][j - 1]);
                        }
                }
        }
        T query(int l, int r) const {
                int len = r - l + 1;
                int j = _log[len];
                return func(table[l][j], table[r - (1 << j) + 1][j]);
        }
private:
        vector<vector<T>> table;
        vector<int> _log;
        func_type func;
};
```

01-Trie

```
struct Trie01 {
	Trie01() : nodes(1) {}
	void insert(ll val) {
		int now = 0;
		for(ll i = 62; i >= 0; --i) {
			ll flag = (val >> i) & 1;
			if(nodes[now].nxt[flag] == -1) {
				nodes[now].nxt[flag] = nodes.size();
				nodes.emplace_back();
			}
			now = nodes[now].nxt[flag];
		}
	}
	ll qmax_xor(ll val) const {
		int now = 0;
		ll ans = 0;
		for(ll i = 62; i >= 0; --i) {
			ll flag = (val >> i) & 1;
			if(nodes[now].nxt[1 ^ flag] != -1) {
				ans += (1ll << i);
				now = nodes[now].nxt[1 ^ flag];
			} else {
				now = nodes[now].nxt[flag];
			}
		}
		return ans;
	}
private:
	static constexpr int height = 63;
	struct Node {
		int nxt[2];
		Node() {
			nxt[0] = nxt[1] = -1;
		}
	};
	vector<Node> nodes;
};
```

CDQ分治

```
struct Data {
        int x, y, z;
        int cnt;
        int ans;
        Data(int x_, int y_, int z_) : x(x_), y(y_), z(z_), cnt(1), ans(-1) {}
};
vector<int> threed_partial(int n, int k, vector<Data> &_datas) {
        Fenwick fwk(k);
        sort(_datas.begin(), _datas.end() - 1, [](const Data &l, const Data &r) {
                if(l.x != r.x) return l.x < r.x;
                if(l.y != r.y) return l.y < r.y;
                return l.z < r.z;
        });
        // 如果值域很大，考虑离散化数据
        vector<Data> datas;
        datas.reserve(n);
        int cnt = 0;
        for(int i = 0; i < n; ++i) {
                ++cnt;
                if((_datas[i].x != _datas[i + 1].x) ||
                        (_datas[i].y != _datas[i + 1].y) ||
                        (_datas[i].z != _datas[i + 1].z)) {
                        datas.emplace_back(_datas[i].x, _datas[i].y, _datas[i].z);
                        cnt = 0;
                }
        }
        int m = datas.size();
        auto cdq = [&](auto &&cdq, int l, int r) -> void {
                if(r - l < 2) {
                        return;
                }
                int mid = (l + r) >> 1;
                cdq(cdq, l, mid);
                cdq(cdq, mid, r);
                sort(datas.begin() + l, datas.begin() + mid, [](const Data &l, const Data &r) {
                        if(l.y != r.y) return l.y < r.y;
                        return l.z < r.z;
                });
                sort(datas.begin() + mid, datas.begin() + r, [](const Data &l, const Data &r) {
                        if(l.y != r.y) return l.y < r.y;
                        return l.z < r.z;
                });
                int j = l;
                for(int i = mid; i < r; ++i) {
                        while(datas[j].y <= datas[i].y && j < mid) {
                                fwk.update(datas[j].z, datas[j].cnt);
                                ++j;
                        }
                        datas[i].ans += fwk.query(datas[i].z);
                }
                for(int k = l; k < j; ++k) {
                        fwk.update(datas[k].z, -datas[k].cnt);
                }
        };
        cdq(cdq, 0, m);
```

```
        vector<int> ans(n, 0);
        for(int i = 0; i < m; ++i) {
                ans[datas[i].ans + datas[i].cnt - 1] += datas[i].cnt;
        }
        return ans;
}
```

莫队

```
struct Query {
        int idx;
        // 左闭右开
        int l, r;
        int ans;
};
// 查询区间内满足元素出现次数等于自身的数的个数
void mo_algo(vector<Query> &queries, const vector<int> &nums) {
        int n = nums.size();
        int len = sqrt(n);
        sort(queries.begin(), queries.end(), [&](const Query &ql, const Query &qr) {
                int atl = ql.l / len, atr = qr.l / len;
                if(atl != atr) {
                        return atl < atr;
                }
                return (bool)((atl % 2 == 0) ^ (ql.r < qr.r));
        });
        int l = 0, r = 0, ans = 0;
        vector<int> count(n + 5, 0);
        auto add = [&](int x) {
                if(x >= n + 5) return;
                if(count[x] == x) --ans;
                count[x]++;
                if(count[x] == x) ++ans;
        };
        auto remove = [&](int x) {
                if(x >= n + 5) return;
                if(count[x] == x) --ans;
                count[x]--;
                if(count[x] == x) ++ans;
        };
        for(auto &q : queries) {
                // 抄板子时注意顺序，先加再更新还是先更新再加
                while(l > q.l) {
                        --l;
                        add(nums[l]);
                }
                while(l < q.l) {
                        remove(nums[l]);
                        ++l;
                }
                while(r < q.r) {
                        add(nums[r]);
                        ++r;
                }
                while(r > q.r) {
                        --r;
                        remove(nums[r]);
                }
                q.ans = ans;
        }
        sort(queries.begin(), queries.end(), [](const Query &l, const Query &r) {
                return l.idx < r.idx;
```

```
        });
}
```

单调栈、单调队列

```
// 单调栈：[l, r)，且有相同元素时，左边取最左端，右边取下一个自己
vector<pair<int, int>> max_range(const vector<int> &nums) {
        int n = nums.size();
        vector ret(n, pair(-1, -1));
        vector<pair<int, int>> stk; // 单调栈
        ret[0].first = 0;
        stk.emplace_back(nums[0], 0);
        for(int i = 1; i < n; ++i) {
                while(!stk.empty() && stk.back().first <= nums[i]) {
                        ret[stk.back().second].second = i;
                        stk.pop_back();
                }
                ret[i].first = stk.empty() ? 0 : stk.back().second + 1;
                stk.emplace_back(nums[i], i);
        }
        for(int i = n - 1; i >= 0; --i) {
                if(ret[i].second == -1) {
                        ret[i].second = n;
                }
        }
        return ret;
}
// 可以有相同元素
        vector ret(n, pair(0, n));
        vector<int> stk;
        for(int i = 0; i < n; ++i) {
                while(!stk.empty() && nums[stk.back()] <= nums[i]) {
                        stk.pop_back();
                }
                if(!stk.empty()) {
                        ret[i].first = stk.back() + 1;
                }
                stk.push_back(i);
        }
        stk.clear();
        for(int i = n - 1; i >= 0; --i) {
                while(!stk.empty() && nums[stk.back()] <= nums[i]) {
                        stk.pop_back();
                }
                if(!stk.empty()) {
                        ret[i].second = stk.back();
                }
                stk.push_back(i);
        }
// 最大滑动窗口：单调队列
vector<int> max_sliding_window(const vector<int> &nums, int k) {
        int n = nums.size();
        vector<int> ret(n - k + 1);
        vector<pair<int, int>> que;
        for(int i = 0; i < k - 1; ++i) {
                while(!que.empty() && que.back().first < nums[i]) {
                        que.pop_back();
                }
                que.emplace_back(nums[i], i);
        }
```

```
        int st = 0;
        for(int i = 0; i <= n - k; ++i) {
                int j = i + k - 1;
                if(que.size() > st && que[st].second < i) {
                        ++st;
                }
                while(!que.empty() && que.back().first < nums[j]) {
                        que.pop_back();
                }
                st = min(st, (int)que.size());
                que.emplace_back(nums[j], j);
                ret[i] = que[st].first;
        }
        return ret;
}
```

图论

Dijkstra

```
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;
vector<ll> dijkstra(vector<vector<pair<int, ll>>> &graph, int start) {
        int v = graph.size();
        vector<ll> dist(v, inf);
        dist[start] = 0;
        vector<bool> visited(v, false);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
        for(auto [vtx, w] : graph[start]) {
                dist[vtx] = w;
                pq.emplace(w, vtx);
        }
        while(!pq.empty()) {
                auto [w, vtx] = pq.top();
                pq.pop();
                if(visited[vtx]) continue;
                visited[vtx] = true;
                for(auto [vt, ww] : graph[vtx]) {
                        if(!visited[vt]) {
                                if(dist[vt] > dist[vtx] + ww) {
                                        dist[vt] = dist[vtx] + ww;
                                        pq.emplace(dist[vt], vt);
                                }
                        }
                }
        }
        return dist;
}
```

Floyd

```
// 操作后，graph就变成了最短路
void floyd(vector<vector<ll>> &graph) {
        for(int k = 0; k < graph.size(); ++k) {
                for(int i = 0; i < graph.size(); ++i) {
                        for(int j = 0; j < graph.size(); ++j) {
                                if(graph[i][j] > graph[i][k] + graph[k][j]) {
                                        graph[i][j] = graph[i][k] + graph[k][j];
                                }
                        }
                }
        }
}
```

SPFA

```
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;
// 返回空vector说明有负环
vector<ll> spfa(const vector<vector<pair<int, ll>>> &graph, int start) {
        int n = graph.size();
        vector<ll> dist(n, inf);
        vector<int> cnt(n, 0);
        vector<bool> inq(n, false);
        queue<int> q;
        q.push(start);
        dist[start] = 0;
        inq[start] = true;
        while(!q.empty()) {
                int u = q.front();
                q.pop();
                inq[u] = false;
                for(auto [v, w] : graph[u]) {
                        if(dist[v] > dist[u] + w) {
                                dist[v] = dist[u] + w;
                                if(!inq[v]) {
                                        inq[v] = true;
                                        q.push(v);
                                        cnt[v] = cnt[u] + 1;
                                        if(cnt[v] > n) return {};
                                }
                        }
                }
        }
        return dist;
}
```

或值最短路

```
int or_mindist(vector<tuple<int, int, int>> &edges, int n) {
        int ans = (1 << 30) - 1;
        for(int i = 29; i >= 0; i--) {
                ans ^= (1 << i);
                DSU dsu(n);
                for(auto &[u, v, w] : edges) {
                        if((w & ans) == w) {
                                dsu.connect(u, v);
                        }
                }
                if(!dsu.is_connected(0, n - 1)) {
                        ans ^= (1 << i);
                }
        }
        return ans;
}
```

Tarjan全家桶

```
// 强连通分量
struct SCC {
        explicit SCC(int n) : nodes(n), graph(n) {}
        void addedge(int u, int v) {
                if(u == v) return;
                graph[u].push_back(v);
        }
        void solve() {
                int dfn = 0, cnt = 0;
                for(int i = 0; i < nodes.size(); i++) {
                        if(nodes[i].dfn == -1) {
                                dfs(dfn, cnt, i);
                        }
                }
                dag.assign(cnt, {});
                set<pair<int, int>> edges;
                for(int u = 0; u < graph.size(); ++u) {
                        for(int v : graph[u]) {
                                int bu = nodes[u].inscc, bv = nodes[v].inscc;
                                if(bu != bv && !edges.contains({ bu, bv })) {
                                        dag[bu].emplace_back(bv);
                                        edges.insert({ bu, bv });
                                }
                        }
                }
        }
        struct Node {
                int dfn;
                int low;
                bool ins;
                int inscc;
                Node() : dfn(-1), low(-1), ins(false), inscc(-1) {}
        };
        vector<Node> nodes;
        vector<vector<int>> graph;
        vector<vector<int>> sccs;
        vector<vector<int>> dag;
private:
        stack<int> scc_stack;
        void dfs(int &dfn, int &cnt, int u) {
                nodes[u].dfn = dfn;
                nodes[u].low = dfn;
                ++dfn;
                scc_stack.push(u);
                nodes[u].ins = true;
                for(int v : graph[u]) {
                        if(nodes[v].dfn == -1) {
                                dfs(dfn, cnt, v);
                                nodes[u].low = min(nodes[u].low, nodes[v].low);
                        } else if(nodes[v].ins) {
                                nodes[u].low = min(nodes[u].low, nodes[v].low);
                        }
                }
                if(nodes[u].dfn == nodes[u].low) {
                        vector<int> scc;
```

```
                        int v = -1;
                        while(v != u) {
                                v = scc_stack.top();
                                scc_stack.pop();
                                nodes[v].ins = false;
                                nodes[v].inscc = cnt;
                                scc.emplace_back(v);
                        }
                        sccs.emplace_back(move(scc));
                        ++cnt;
                }
        }
};
// 边双
struct EBCC {
        explicit EBCC(int n) : nodes(n), graph(n), in_ebcc(n) {}
        void addedge(int u, int v) {
                if(u == v)
                        return;
                graph[u].emplace_back(v);
                graph[v].emplace_back(u);
        }
        void solve() {
                int dfn = 0;
                for(int i = 0; i < nodes.size(); ++i) {
                        if(nodes[i].dfn == -1) {
                                dfs(i, -1, dfn);
                        }
                }
                for(int i = 0; i < ebcc.size(); ++i) {
                        for(int u : ebcc[i]) {
                                in_ebcc[u] = i;
                        }
                }
        }
        vector<vector<int>> graph;
        vector<vector<int>> ebcc;
        vector<int> in_ebcc;
private:
        struct Node {
                int dfn;
                int low;
                bool ins;
                Node() : dfn(-1), low(-1), ins(false) {}
        };
        vector<Node> nodes;
        stack<int> stk;
        void dfs(int u, int fa, int &dfn) {
                nodes[u].dfn = nodes[u].low = dfn;
                nodes[u].ins = true;
                dfn++;
                stk.push(u);
                for(int v : graph[u]) {
                        if(v == fa)
                                continue;
                        if(nodes[v].dfn == -1) {
                                dfs(v, u, dfn);
```

```
                                nodes[u].low = min(nodes[u].low, nodes[v].low);
                        } else if(nodes[v].ins) {
                                nodes[u].low = min(nodes[u].low, nodes[v].dfn);
                        }
                }
                if(nodes[u].dfn == nodes[u].low) {
                        vector<int> t;
                        int n = -1;
                        while(n != u) {
                                n = stk.top();
                                stk.pop();
                                t.emplace_back(n);
                                nodes[n].ins = false;
                        }
                        ebcc.emplace_back(move(t));
                }
        }
};
// 点双
struct DCC {
        explicit DCC(int n)
                : nodes(n), graph(n) {}
        void addedge(int u, int v) {
                if(u == v) return;
                graph[u].emplace_back(v);
                graph[v].emplace_back(u);
        }
        void solve() {
                int dfn_now = 0;
                for(int i = 0; i < nodes.size(); ++i) {
                        if(nodes[i].dfn == -1) {
                                dfs(i, -1, i, dfn_now);
                        }
                }
        }
        struct Node {
                int dfn;
                int low;
                Node() :dfn(-1), low(-1) {}
        };
        vector<Node> nodes;
        vector<vector<int>> graph;
        vector<vector<int>> dcc;
private:
        stack<int> stk;
        void dfs(int u, int parent, int root, int &dfn_now) {
                nodes[u].dfn = dfn_now;
                nodes[u].low = dfn_now;
                dfn_now++;
                stk.push(u);
                int child = 0;
                for(int v : graph[u]) {
                        if(v == parent) continue;
                        if(nodes[v].dfn == -1) {
                                dfs(v, u, root, dfn_now);
                                nodes[u].low = min(nodes[u].low, nodes[v].low);
                                if(nodes[v].low >= nodes[u].dfn) {
```

```
                                        ++child;
                                        vector<int> f = { u };
                                        while(!stk.empty()) {
                                                int x = stk.top();
                                                stk.pop();
                                                f.emplace_back(x);
                                                if(x == v) break;
                                        }
                                        dcc.emplace_back(move(f));
                                }
                        } else {
                                nodes[u].low = min(nodes[u].low, nodes[v].dfn);
                        }
                }
                if(u == root && graph[u].empty()) {
                        dcc.emplace_back(vector<int>{u});
                        return;
                }
        }
};
// 割点
struct Cutpoint {
        explicit Cutpoint(int n) : nodes(n), graph(n) {}
        void addedge(int u, int v) {
                graph[u].emplace_back(v);
                graph[v].emplace_back(u);
        }
        void solve() {
                int dfn = 0;
                for(int i = 0; i < nodes.size(); ++i) {
                        if(nodes[i].dfn == -1) {
                                dfs(i, -1, dfn);
                        }
                }
                sort(cutpoints.begin(), cutpoints.end());
                cutpoints.erase(unique(cutpoints.begin(), cutpoints.end()), cutpoints.end());
        }
        struct Node {
                int dfn, low;
                Node() : dfn(-1), low(-1) {}
        };
        vector<Node> nodes;
        vector<vector<int>> graph;
        vector<int> cutpoints;
private:
        void dfs(int u, int fa, int &dfn) {
                nodes[u].dfn = nodes[u].low = dfn;
                ++dfn;
                int child = 0;
                bool flag = false;
                for(int v : graph[u]) {
                        if(nodes[v].dfn == -1) {
                                ++child;
                                dfs(v, u, dfn);
                                nodes[u].low = min(nodes[u].low, nodes[v].low);
                                if(fa != -1) {
                                        flag |= (nodes[v].low >= nodes[u].dfn);
```

```
                                }
                        } else if(v != fa) {
                                nodes[u].low = min(nodes[u].low, nodes[v].dfn);
                        }
                }
                if(fa == -1) {
                        flag = (child > 1);
                }
                if(flag) {
                        cutpoints.emplace_back(u);
                }
        }
};
// 桥
struct Bridge {
        explicit Bridge(int n)
                : nodes(n), graph(n) {}
        void addedge(int u, int v) {
                graph[u].emplace_back(v);
                graph[v].emplace_back(u);
        }
        void solve() {
                int dfn_now = 0;
                for(int i = 0; i < nodes.size(); ++i) {
                        if(nodes[i].dfn == -1) {
                                dfs(i, -1, dfn_now);
                        }
                }
        }
        struct Node {
                int dfn;
                int low;
                Node() : dfn(-1), low(-1) {}
        };
        vector<Node> nodes;
        vector<vector<int>> graph;
        vector<pair<int, int>> bridges;
private:
        void dfs(int u, int father, int &dfn_now) {
                nodes[u].dfn = nodes[u].low = dfn_now++;
                for(int v : graph[u]) {
                        if(nodes[v].dfn == -1) {
                                dfs(v, u, dfn_now);
                                nodes[u].low = min(nodes[u].low, nodes[v].low);
                                if(nodes[v].low > nodes[u].dfn) {
                                        bridges.emplace_back(min(u, v), max(u, v));
                                }
                        } else if(v != father) {
                                nodes[u].low = min(nodes[u].low, nodes[v].dfn);
                        }
                }
        }
};
```

同余最短路

```
// 跑的dijkstra
ll remainder_mindist(ll k, vector<ll> dist) {
        ll modulo = 2 * min(dist[0], dist[3]);
        vector<vector<ll>> dp(4, vector<ll>(modulo, inf));
        vector<vector<bool>> visited(4, vector<bool>(modulo, false));
        dp[0][0] = 0;
        priority_queue<tuple<ll, ll, ll>, vector<tuple<ll, ll, ll>>, greater<>> pq;
        pq.emplace(0, 0, 0);
        auto get = [](ll a, ll b) -> ll {
                if(a == 3 && b == 0) {
                        return 3;
                }
                if(b == 3 && a == 0) {
                        return 3;
                }
                return min(a, b);
        };
        while(!pq.empty()) {
                auto [w, m, c] = pq.top();
                pq.pop();
                if(visited[c][m]) continue;
                visited[c][m] = true;
                ll d1 = (c + 1) % 4, d2 = (c + 3) % 4;
                ll w1 = dist[get(c, d1)], w2 = dist[get(c, d2)];
                if(chkmin(dp[d1][(w + w1) % modulo], w + w1)) {
                        pq.emplace(w + w1, (w + w1) % modulo, d1);
                }
                if(chkmin(dp[d2][(w + w2) % modulo], w + w2)) {
                        pq.emplace(w + w2, (w + w2) % modulo, d2);
                }
        }
        ll ans = inf;
        for(ll i = 0; i < modulo; ++i) {
                ll required = (k / modulo) * modulo + i;
                while(required < k) {
                        required += modulo;
                }
                chkmin(ans, max(required, dp[0][i]));
        }
        return ans;
}
```

Kruskal

```cpp
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;
struct Edge {
        int u, v;
        ll w;
};
// 拿个并查集板子下来谢谢喵
ll kruskal(vector<Edge> &edges, int n) {
        DSU dsu(n);
        ll ans = 0;
        sort(edges.begin(), edges.end(), [](const Edge &a, const Edge &b) {
                return a.w < b.w;
        });
        for(auto &e : edges) {
                if(!dsu.is_connected(e.u, e.v)) {
                        dsu.connect(e.u, e.v);
                        ans += e.w;
                }
        }
        for(int i = 1; i < n; ++i) {
                if(!dsu.is_connected(i, 0)) {
                        return inf;
                }
        }
        return ans;
}
```

Prim

```
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;
ll prim(const vector<vector<pair<int, ll>>> &graph) {
        int n = graph.size();
        vector<bool> visited(n, false);
        vector<ll> dist(n, inf);
        dist[0] = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
        pq.emplace(0, 0);
        ll ret = 0;
        while(!pq.empty()) {
                auto [w, v] = pq.top();
                pq.pop();
                if(visited[v]) continue;
                visited[v] = true;
                ret += w;
                for(auto [u, w] : graph[v]) {
                        if(!visited[u] && dist[u] > w) {
                                dist[u] = w;
                                pq.emplace(w, u);
                        }
                }
        }
        for(bool b : visited) {
                if(!b) {
                        return inf;
                }
        }
        return ret;
}
```

Boruvka

```
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;
struct Edge {
        int u, v;
        ll w;
};
ll boruvka(const vector<Edge> &edges, int n) {
        DSU dsu(n);
        ll ans = 0;
        int c = n;
        vector<Edge> mst(n);
        while(c > 1) {
                for(int i = 0; i < n; ++i) {
                        if(dsu.find(i) != i) continue;
                        mst[i] = Edge(-1, -1, inf);
                }
                for(auto [u, v, w] : edges) {
                        int fu = dsu.find(u), fv = dsu.find(v);
                        if(fu == fv) continue;
                        if(mst[fu].w > w) {
                                mst[fu] = Edge(u, v, w);
                        }
                        if(mst[fv].w > w) {
                                mst[fv] = Edge(u, v, w);
                        }
                }
                bool flag = false;
                for(int i = 0; i < n; ++i) {
                        if(dsu.find(i) != i) continue;
                        auto [u, v, w] = mst[i];
                        if(w == inf) return inf;
                        if(!dsu.is_connected(u, v)) {
                                dsu.connect(u, v);
                                ans += w;
                                --c;
                                flag = true;
                        }
                }
                if(!flag) {
                        return inf;
                }
        }
        return ans;
}
```

Dinic

```
constexpr ll inf = 0x3f3f3f3f3f3f3f3f;
struct Dinic {
	Dinic(int n, int s, int e)
		: graph(n), level(n), start(s), end(e), n(n) {}
	void addedge(int u, int v, ll w) {
		graph[u].emplace_back(Edge{ v, (int)graph[v].size(), w });
		graph[v].emplace_back(Edge{ u, (int)graph[u].size() - 1, 0 });
	}
	ll solve() {
		ll ans = 0;
		while(bfs()) {
			ll flow = dfs(start, inf);
			while(flow > 0) {
				ans += flow;
				flow = dfs(start, inf);
			}
		}
		return ans;
	}
private:
	struct Edge {
		int to;
		int rev;
		ll cap;
	};
	vector<vector<Edge>> graph;
	vector<int> level;
	int n;
	int start, end;
	bool bfs() {
		fill(level.begin(), level.end(), -1);
		level[start] = 0;
		queue<int> q;
		q.emplace(start);
		while(!q.empty()) {
			int u = q.front();
			q.pop();
			for(const auto &e : graph[u]) {
				if(level[e.to] == -1 && e.cap > 0) {
					level[e.to] = level[u] + 1;
					q.emplace(e.to);
				}
			}
		}
		return level[end] != -1;
	}
	ll dfs(int u, ll max_flow) {
		if(u == end || max_flow == 0) return max_flow;
		ll total_flow = 0;
		for(auto &e : graph[u]) {
			if(level[e.to] == level[u] + 1 && e.cap > 0) {
				ll min_flow = min(max_flow, e.cap);
				ll pushed = dfs(e.to, min_flow);
				if(pushed > 0) {
					e.cap -= pushed;
```

```
                                        graph[e.to][e.rev].cap += pushed;
                                        total_flow += pushed;
                                        max_flow -= pushed;
                                        if(max_flow == 0) {
                                                return total_flow;
                                        }
                                }
                        }
                        return total_flow;
        }
};
```

最小费用最大流

EK

```
struct EK {
    struct Edge {
        int dst, rev;
        ll weight, flow;
        bool neg;
    };
    vector<vector<Edge>> graph;
    vector<int> pv, pe;
    vector<ll> mf, d;
    vector<bool> inq;
    int n;

    explicit EK(int _n) : n(_n), graph(_n), pv(_n, -1), pe(_n, -1), d(_n, inf), mf(_n, 0), inq(_n, false) {}

    void addedge(int u, int v, ll flow, ll weight) {
        int iu = graph[u].size(), iv = graph[v].size();
        graph[u].push_back({ v, iv, weight, flow, false });
        graph[v].push_back({ u, iu, -weight, 0, true });
    }

    pair<ll, ll> solve() {
        ll cost = 0, flow = 0;
        while(true) {
            for(int i = 0; i < n; ++i) {
                d[i] = inf;
                mf[i] = 0;
                inq[i] = false;
                pv[i] = -1;
                pe[i] = -1;
            }
            queue<int> q;
            q.push(0);
            d[0] = 0;
            mf[0] = inf;
            inq[0] = true;
            while(!q.empty()) {
                int u = q.front();
                q.pop();
                inq[u] = false;
                for(int i = 0; i < graph[u].size(); ++i) {
                    auto &e = graph[u][i];
                    int v = e.dst;
                    ll f = e.flow, w = e.weight;
                    if(f > 0 && d[v] > d[u] + w) {
                        d[v] = d[u] + w;
                        pv[v] = u;
                        pe[v] = i;
                        mf[v] = min(mf[u], f);
                        if(!inq[v]) {
                            q.push(v);
                            inq[v] = true;
                        }
                    }
```

```
                }
            }
            if(mf[n - 1] <= 0) break;
            ll add = mf[n - 1];
            for(int v = n - 1; v != 0; v = pv[v]) {
                int u = pv[v], e = pe[v];
                graph[u][e].flow -= add;
                graph[v][graph[u][e].rev].flow += add;
            }
            flow += add;
            cost += add * d[n - 1];
        }
        return { cost, flow };
    }
};
```

匈牙利算法（不带权）

```
int hungarian(const vector<vector<int>> &graph, int vsz) {
        int usz = graph.size();
        vector<int> mu(usz, -1);
        vector<int> mv(vsz, -1);
        auto dfs = [&](auto &&dfs, int u, vector<bool> &visited) -> bool {
                for(int v : graph[u]) {
                        if(visited[v]) continue;
                        visited[v] = true;
                        if(mv[v] == -1 || dfs(dfs, mv[v], visited)) {
                                mv[v] = u;
                                mu[u] = v;
                                return true;
                        }
                }
                return false;
        };
        int ret = 0;
        for(int u = 0; u < usz; ++u) {
                if(mu[u] == -1) {
                        vector<bool> visited(vsz, false);
                        if(dfs(dfs, u, visited)) {
                                ret++;
                        }
                }
        }
        return ret;
}
```

2-SAT

```cpp
// 本题给的是析取式，在这里我转化为了蕴含式求解
// 请打一个SCC下来
// 注意：蕴含式也要加另一条边，A->B要加B'->A'
vector<int> solve_twosat(int n, const vector<tuple<int, int, int, int>> &conds) {
        SCC solver(2 * n);
        for(auto [i, flag_i, j, flag_j] : conds) {
                solver.addedge(2 * i + 1 - flag_i, 2 * j + flag_j);
                solver.addedge(2 * j + 1 - flag_j, 2 * i + flag_i);
        }
        solver.solve();
        for(int i = 0; i < n; ++i) {
                if(solver.nodes[2 * i].inscc == solver.nodes[2 * i + 1].inscc) {
                        return {};
                }
        }
        vector<int> ans(n, -1);
        for(int i = 0; i < solver.sccs.size(); ++i) {
                for(int ii : solver.sccs[i]) {
                        int u = ii / 2;
                        int v = ii % 2;
                        if(ans[u] == -1) {
                                ans[u] = v;
                        }
                }
        }
        return ans;
}
```

搜索

迭代加深

```
pair<int, stack<int>> iddfs(array<array<int, 3>, 3> src, const array<array<int, 3>, 3> &dst) {
	constexpr int dx[] = { -1, 1, 0, 0 };
	constexpr int dy[] = { 0, 0, -1, 1 };
	int ans = 0;
	stack<int> stk;
	pair<int, int> zeropos;
	for(int i = 0; i < 3; ++i) {
		for(int j = 0; j < 3; ++j) {
			if(src[i][j] == 0) {
				zeropos = { i, j };
			}
		}
	}
	auto dfs = [&](auto &&dfs, int depth, int lastdir) -> bool {
		if(src == dst) {
			for(int i = 0; i < 3; ++i) {
				for(int j = 0; j < 3; ++j) {
					stk.push(src[i][j]);
				}
			}
			return true;
		}
		if(depth >= ans) return false;
		auto revdir = [](int cur) -> int {
			if(cur < 2) return 1 - cur;
			return 5 - cur;
		};
		for(int nowdir = 0; nowdir < 4; ++nowdir) {
			if(nowdir == revdir(lastdir)) {
				continue;
			}
			auto [oldx, oldy] = zeropos;
			int newx = oldx + dx[nowdir], newy = oldy + dy[nowdir];
			if(newx < 0 || newx > 2 || newy < 0 || newy > 2) continue;
			swap(src[oldx][oldy], src[newx][newy]);
			zeropos = { newx, newy };
			if(dfs(dfs, depth + 1, nowdir)) {
				swap(src[oldx][oldy], src[newx][newy]);
				zeropos = { oldx, oldy };
				for(int i = 0; i < 3; ++i) {
					for(int j = 0; j < 3; ++j) {
						stk.push(src[i][j]);
					}
				}
				return true;
			}
			swap(src[oldx][oldy], src[newx][newy]);
			zeropos = { oldx, oldy };
		}
		return false;
	};
	for(ans = 0; ans < 33; ++ans) {
```

```
                if(dfs(dfs, 0, 114514)) {
                        return { ans, stk };
                }
        }
        return { -1, stack<int>{} };
}
```

动态规划

背包DP

```
namespace OneD {
// 01背包
ll knapsack_01(const vector<pair<int, ll>> &objects, int maxw) {
        int n = objects.size();
        vector<ll> dp(maxw + 1, 0);
        for(auto [w, v] : objects) {
                for(int j = maxw; j >= w; --j) {
                        dp[j] = max(dp[j], dp[j - w] + v);
                }
        }
        return dp[maxw];
}
// 完全背包
ll knapsack_complete(const vector<pair<int, ll>> &objects, int maxw) {
        int n = objects.size();
        vector<ll> dp(maxw + 1, 0);
        for(auto [w, v] : objects) {
                for(int j = w; j <= maxw; ++j) {
                        dp[j] = max(dp[j], dp[j - w] + v);
                }
        }
        return dp[maxw];
}
struct Object {
        int cost, cnt;
        ll w;
};
// 多重背包
ll multi_knapsack(const vector<Object> &objects, int maxw) {
        vector<ll> dp(maxw + 1, 0);
        for(auto [cost, cnt, w] : objects) {
                for(int _ = 0; _ < cnt; ++_) {
                        for(int j = maxw; j >= cost; --j) {
                                dp[j] = max(dp[j], dp[j - cost] + w);
                        }
                }
        }
        return dp[maxw];
}
// 多重背包的二进制优化
ll multi_knapsack_binary(const vector<Object> &objects, int maxw) {
        vector<pair<int, ll>> objs;
        for(auto [cost, cnt, w] : objects) {
                int k = 1;
                while(cnt > 0) {
                        int take = min(k, cnt);
                        objs.emplace_back(cost * take, w * take);
                        cnt -= take;
                        k *= 2;
                }
        }
        return knapsack_01(objs, maxw);
```

```
}
// 缺少：多重背包的单调队列优化
}

namespace TwoD {
struct Object {
        int u, v;
        ll w;
};
// 二维01背包
ll knapsack_01(int U, int V, const vector<Object> &objects) {
        vector<vector<ll>> dp(U + 1, vector<ll>(V + 1, 0));
        for(auto [u, v, w] : objects) {
                for(int i = U; i - u >= 0; --i) {
                        for(int j = V; j - v >= 0; --j) {
                                dp[i][j] = max(dp[i][j], dp[i - u][j - v] + w);
                        }
                }
        }
        return dp[U][V];
}

}
```

区间DP

```
int main() {
        cin.tie(nullptr)->sync_with_stdio(false);
        int n;
        cin >> n;
        vector<ll> nums(n);
        for(int i = 0; i < n; ++i) {
                cin >> nums[i];
        }
        vector<int> s(2 * n + 1);
        for(int i = 0; i < 2 * n; ++i) {
                s[i + 1] = s[i] + nums[i];
        }
        vector<vector<int>> dp(2 * n + 1, vector<int>(2 * n + 1));
        for(int len = 2; len <= n; ++len) {
                for(int i = 0; i <= 2 * n - len; ++i) {
                        int j = i + len;
                        int mx = 0;
                        for(int k = i + 1; k < j; ++k) {
                                mx = max(mx, dp[i][k] + dp[k][j]);
                        }
                        dp[i][j] = mx + s[j] - s[i];
                }
        }
        int ans = 0;
        for(int i = 0; i < n; ++i) {
                ans = max(ans, dp[i][i + n]);
        }
        cout << ans << '\n';
        return 0;
}
```

数位DP

```
constexpr ll modulo = 10'0000'0007;
ll digit_dp(string k, int d) {
        vector<vector<ll>> memo(k.size(), vector<ll>(d, -1));
        auto dfs = [&](auto &&dfs, int idx, ll psum, bool isnum, bool islimit) -> ll {
                if(idx == k.size()) {
                        return (ll)(isnum && (psum == 0));
                }
                if(memo[idx][psum] != -1 && isnum && !islimit) {
                        return memo[idx][psum];
                }
                ll ans = 0;
                if(!isnum) {
                        ans = (ans + dfs(dfs, idx + 1, 0, false, false)) % modulo;
                }
                int llim = isnum ? 0 : 1, hlim = islimit ? (k[idx] - '0') : 9;
                for(int i = llim; i <= hlim; ++i) {
                        ans = (ans + dfs(dfs, idx + 1, ((psum + i) % d), true, (islimit && (i == hlim)))) % modulo;
                }
                if(isnum && !islimit) {
                        memo[idx][psum] = ans;
                }
                return ans;
        };
        return dfs(dfs, 0, 0, false, true);
}
```

计数DP

```
constexpr ll modulo = 10'0000'0007;
ll count_dp(string s) {
        int n = s.size();
        vector<vector<ll>> dp(n, vector<ll>(n, 0));
        dp[0][0] = 1;
        vector<vector<ll>> sum(n, vector<ll>(n + 1, 0));
        sum[0][1] = 1;
        for(int i = 1; i < n; ++i) {
                for(int j = 0; j <= i; ++j) {
                        if(s[i - 1] == '<') {
                                dp[i][j] = sum[i - 1][j];
                        } else {
                                dp[i][j] = (sum[i - 1][i] - sum[i - 1][j] + modulo) % modulo;
                        }
                }
                sum[i][0] = 0;
                for(int j = 1; j <= i + 1; ++j) {
                        sum[i][j] = (sum[i][j - 1] + dp[i][j - 1]) % modulo;
                }
        }
        ll ans = 0;
        for(int j = 0; j < n; ++j) {
                ans = (ans + dp[n - 1][j]) % modulo;
        }
}
```

线段树优化DP

```
// SegTree区间最大值、单点加法线段树
int lis(const vector<int> &nums) {
        auto discrete = nums;
        sort(discrete.begin(), discrete.end());
        discrete.erase(unique(discrete.begin(), discrete.end()), discrete.end());
        unordered_map<int, int> mp;
        for(int i = 0; i < discrete.size(); i++) {
                mp[discrete[i]] = i;
        }
        SegTree seg(discrete.size());
        for(int i = 0; i < nums.size(); i++) {
                int x = mp[nums[i]];
                int v = seg.query(0, x) + 1;
                seg.update(x, v);
        }
        return seg.query(0, discrete.size());
}
```

数学

快速幂和模整数类

```
ll qpow(ll x, ll n) {
	ll ret = 1;
	for(; n != 0; n >>= 1, x = x * x % modulo) {
		if(n & 1) ret = ret * x % modulo;
	}
	return ret;
}

struct PreprocessedPow {
	PreprocessedPow(ll k, ll maxn) {
		k %= modulo;
		m = ceil(sqrt(maxn + 1.5));
		powerk.assign(m, 1);
		powerkm.assign(m, 1);
		for(int i = 1; i < m; ++i) {
			powerk[i] = (ll(powerk[i - 1]) * k) % modulo;
		}
		powerkm[1] = ll(powerk[m - 1]) * k % modulo;
		for(int i = 2; i < m; ++i) {
			powerkm[i] = ll(powerkm[i - 1]) * ll(powerkm[1]) % modulo;
		}
	}
	ll pow(ll n) const {
		ll i = n / m, j = n % m;
		return (ll(powerkm[i]) * ll(powerk[j])) % modulo;
	}
private:
	int m;
	vector<int> powerk, powerkm;
};

struct ModInt {
	ModInt(ll v = 0) : val(v % modulo) {
		if(val < 0) val += modulo;
	}
	ModInt operator+(const ModInt &rhs) const {
		return ModInt(val + rhs.val);
	}
	ModInt operator-(const ModInt &rhs) const {
		return ModInt(val - rhs.val);
	}
	ModInt operator*(const ModInt &rhs) const {
		return ModInt(val * rhs.val);
	}
	ModInt operator/(const ModInt &rhs) const {
		return ModInt(val * qpow(rhs.val, modulo - 2));
	}
	ModInt power(int n) const {
		ModInt ret = 1;
		for(ModInt base = val; n != 0; n >>= 1, base = base * base) {
			if(n & 1) ret = ret * base;
		}
```

```
                return ret;
        }
        operator ll() const {
                return val;
        }
private:
        ll val;
};
```

组合数

```
struct Comb {
        Comb() = delete;
        static ll fact(ll n) {
                return _fact[n];
        }
        static ll invfact(ll n) {
                return _invfact[n];
        }
        static ll comb(ll n, ll m) {
                if(m < 0 || m > n) return 0;
                return (((ll(_fact[n]) * ll(_invfact[m])) % modulo) * ll(_invfact[n - m])) % modulo;
        }
private:
        static constexpr int _maxn = 500005;
        static inline int _fact[_maxn], _invfact[_maxn];
        static inline int init = [&] {
                _fact[0] = 1;
                for(ll i = 1; i < _maxn; ++i) {
                        _fact[i] = (ll(_fact[i - 1]) * i) % modulo;
                }
                _invfact[_maxn - 1] = qpow(_fact[_maxn - 1], modulo - 2);
                for(ll i = _maxn - 2; i >= 0; --i) {
                        _invfact[i] = (ll(_invfact[i + 1]) * (i + 1)) % modulo;
                }
                return 0;
        }();
};
```

EXGCD

```
// @returns (gcd, x, y) so that gcd = ax + by
tuple<ll, ll, ll> exgcd(ll a, ll b) {
        if(b == 0) return tuple(a, 1ll, 0ll);
        auto [g, x, y] = exgcd(b, a % b);
        return tuple(g, y, x - (a / b) * y);
}
```

中国剩余定理

```
// @returns (a, b) so that answer is a + kb, k\in N_+
pair<ll, ll> crt(const vector<ll> &rem, const vector<ll> &mod) {
        int n = rem.size();
        ll modulo_ = 1, ans = 0;
        for(int i = 0; i < n; ++i) {
                modulo_ *= mod[i];
        }
        for(int i = 0; i < n; ++i) {
                ll m = modulo_ / mod[i];
                auto [_, b, __] = exgcd(m, mod[i]);
                ans = (ans + ((rem[i] * m) % modulo_ * b) % modulo_) % modulo_;
        }
        return pair((ans % modulo_ + modulo_) % modulo_, modulo_);
}

pair<ll, ll> excrt(const vector<ll> &rem, const vector<ll> &mod) {
        int n = rem.size();
        ll r1 = rem[0], m1 = mod[0];
        for(int i = 1; i < n; ++i) {
                ll r2 = rem[i], m2 = mod[i];
                auto [g, p, _] = exgcd(m1, m2);
                if((r2 - r1) % g != 0) {
                        return { -1, -1 };
                }
                ll v = m2 / g, x = (r2 - r1) / g;
                ll u = p % v * x % v;
                ll w = (u % v + v) % v;
                r1 += w * m1;
                m1 = lcm(m1, m2);
        }
        return { (r1 % m1 + m1) % m1, m1 };
}
```

整除分块

```
ll intdiv(ll n) {
    ll ans = 0;
        for(ll b = 1; b <= n;) {
                ll val = n / b;
                ll r = n / val;
                ll len = r - b + 1;
                ans = (ans + (len % modulo * val % modulo)) % modulo;
                b = r + 1;
        }
    return ans;
}
```

二项式反演公式

$$f(n)=\sum_{k=0}^{n}\binom{n}{k}g(k)\to g(n)=\sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}f(k)$$

$$f(n)=\sum_{k=n}^{m}\binom{k}{n}g(k)\rightarrow g(n)=\sum_{k=n}^{m}(-1)^{k-n}\binom{k}{n}f(k)$$

斯特林数

预处理

```
struct Sterling {
        Sterling() = delete;
        static int get(int n, int m) {
                return ster[n][m];
        }
private:
        static constexpr int maxn = 5005;
        static inline int ster[maxn][maxn];
        static inline int init = [&] {
                ster[0][0] = 1;
                for(ll i = 1; i < maxn; ++i) {
                        ster[i][0] = 0;
                        for(ll j = 1; j < i; ++j) {
                                ster[i][j] = (ll(ster[i - 1][j - 1]) + ll(ster[i - 1][j]) * j) % modulo;
                        }
                }
                return 0;
        }();
};
```

使用NTT求一行

得到的是 $\left\{ {0 \atop k} \right\},\left\{ {1 \atop k} \right\},\ldots,\left\{ {k \atop k} \right\}$

```cpp
vector<ll> sterling_ntt(int k) {
        if(k == 0) return {1};
        vector<ll> powk(k + 1, 0);
        powk[1] = 1;
        vector<bool> isprime(k + 1, true);
        isprime[0] = isprime[1] = false;
        vector<int> primes;
        for(int i = 2; i <= k; ++i) {
                if(isprime[i]) {
                        primes.push_back(i);
                        powk[i] = qpow(i, k);
                }
                for(ll p : primes) {
                        ll q = i * p;
                        if(q > k) break;
                        isprime[q] = false;
                        powk[q] = powk[i] * powk[p] % modulo;
                        if(i % p == 0) break;
                }
        }
        vector<ll> fact(k + 1), invfact(k + 1);
        fact[0] = fact[1] = 1;
        for(ll i = 2; i <= k; ++i) {
                fact[i] = fact[i - 1] * i % modulo;
        }
        invfact[k] = qpow(fact[k], modulo - 2);
        for(int i = k - 1; i >= 0; --i) {
                invfact[i] = invfact[i + 1] * (i + 1) % modulo;
        }
        int n = 1;
        while(n < (2 * k + 1)) {
                n <<= 1;
        }
        vector<ll> a(n, 0), b(n, 0);
        for(int i = 0; i <= k; ++i) {
                a[i] = powk[i] * invfact[i] % modulo;
                if(i % 2 == 1) {
                        b[i] = (modulo - invfact[i]) % modulo;
                } else {
                        b[i] = invfact[i];
                }
        }
        auto ntt = [](auto &&ntt, vector<ll> &f, bool invert) -> void {
                int n = f.size();
                if(n == 1) return;
                vector<ll> f0(n / 2), f1(n / 2);
                for(int i = 0; i < n / 2; ++i) {
                        f0[i] = f[2 * i];
                        f1[i] = f[2 * i + 1];
                }
                ntt(ntt, f0, invert);
                ntt(ntt, f1, invert);
                ll w = 1, wn = qpow(g, (modulo - 1) / n);
                if(invert) {
                        wn = qpow(wn, modulo - 2);
                }
                for(int i = 0; i < n / 2; ++i) {
```

```
                        ll u = f0[i];
                        ll v = w * f1[i] % modulo;
                        f[i] = (u + v) % modulo;
                        f[i + n / 2] = (u - v + modulo) % modulo;
                        w = w * wn % modulo;
                }
        };
        ntt(ntt, a, false);
        ntt(ntt, b, false);
        vector<ll> c(n);
        for(int i = 0; i < n; ++i) {
                c[i] = a[i] * b[i] % modulo;
        }
        ntt(ntt, c, true);
        ll invn = qpow(n, modulo - 2);
        for(int i = 0; i < n; ++i) {
                c[i] = c[i] * invn % modulo;
        }
        c.resize(k + 1);
        return c;
}
```

质数筛和它能干的事

```
ll __qpow(ll x, ll n) {
	ll ret = 1;
	for(; n != 0; n >>= 1, x = x * x) {
		if(n & 1) ret = ret * x;
	}
	return ret;
}

template<class F> concept eularsieve_func = convertible_to<F, function<ll(int, int)>>;

struct EularSieve {
	vector<int> primes, lpf, lpow;
	vector<ll> fv;

	explicit EularSieve(int n) : lpf(n + 1), lpow(n + 1) {
		for(ll i = 2; i <= n; ++i) {
			if(lpf[i] == 0) {
				lpf[i] = i;
				lpow[i] = 1;
				primes.push_back(i);
			}
			for(ll p : primes) {
				ll j = i * p;
				if(j > n) break;
				lpf[j] = p;
				if(i % p == 0) {
					lpow[j] = lpow[i] + 1;
					ll jp = __qpow(lpf[j], lpow[j]);
					ll rem = j / jp;
					break;
				} else {
					lpow[j] = 1;
				}
			}
		}
	}

	template<eularsieve_func F> EularSieve(int n, F &&f) : lpf(n + 1), lpow(n + 1), fv(n + 1) {
		fv[1] = 1;
		for(ll i = 2; i <= n; ++i) {
			if(lpf[i] == 0) {
				lpf[i] = i;
				lpow[i] = 1;
				primes.push_back(i);
				fv[i] = f(i, 1);
			}
			for(ll p : primes) {
				ll j = i * p;
				if(j > n) break;
				lpf[j] = p;
				if(i % p == 0) {
					lpow[j] = lpow[i] + 1;
					ll jp = __qpow(lpf[j], lpow[j]);
					ll rem = j / jp;
					if(rem == 1) {
```

```
                                        fv[j] = f(p, lpow[j]);
                                } else {
                                        fv[j] = fv[rem] * fv[jp];
                                }
                                break;
                        } else {
                                fv[j] = fv[i] * fv[p];
                                lpow[j] = 1;
                        }
                }
        }
    }
};

int eular_f(int p, int k) {
    return __qpow(p, k) - __qpow(p, k - 1);
}
int mobius_f(int p, int k) {
    return k == 0 ? 1 : k == 1 ? -1 : 0;
}
int factor_cnt_f(int p, int k) {
    return k + 1;
}
int factor_sum_f(int p, int k) {
    return (__qpow(p, k + 1) - 1ll) / ll(p - 1);
}
```

莫比乌斯反演公式

$$F(n)=\sum_{d|n}f(d)\leftrightarrow f(n)=\sum_{d|n}\mu(d)F(\frac{n}{d})$$

$$f*1=F\leftrightarrow f=F*\mu$$

多项式乘法

FFT

```
using cd = complex<double>;
constexpr double pi = 3.14159265358979323846264338327950288;
vector<int> multiply(const vector<int> &a, const vector<int> &b) {
        int n = 1;
        while(n < (a.size() + b.size())) {
                n <<= 1;
        }
        vector<cd> fa(n), fb(n);
        for(int i = 0; i < a.size(); ++i) {
                fa[i] = a[i];
        }
        for(int i = 0; i < b.size(); ++i) {
                fb[i] = b[i];
        }
        auto fft = [](auto &&fft, vector<cd> &f, bool invert) -> void {
                int n = f.size();
                if(n == 1) return;
                vector<cd> f0(n / 2), f1(n / 2);
                for(int i = 0; i < n / 2; ++i) {
                        f0[i] = f[2 * i];
                        f1[i] = f[2 * i + 1];
                }
                fft(fft, f0, invert);
                fft(fft, f1, invert);
                double theta = 2.1 * pi / n * (invert ? -1.1 : 1.1);
                cd wt = 1, w(cos(theta), sin(theta));
                for(int t = 0; t < n / 2; ++t) {
                        cd u = f0[t], v = wt * f1[t];
                        f[t] = u + v;
                        f[t + n / 2] = u - v;
                        wt *= w;
                }
        };
        fft(fft, fa, false);
        fft(fft, fb, false);
        for(int i = 0; i < n; ++i) {
                fa[i] *= fb[i];
        }
        fft(fft, fa, true);
        for(int i = 0; i < n; ++i) {
                fa[i] /= n;
        }
        vector<int> ret(n);
        for(int i = 0; i < n; ++i) {
                ret[i] = int(fa[i].real() + (fa[i].real() > 0.1 ? 0.51 : -0.51));
        }
        while(ret.size() > (a.size() + b.size() - 1)) {
                ret.pop_back();
        }
        return ret;
}
```

NTT

```
constexpr ll modulo = 9'9824'4353, g = 3;
// constexpr ll modulo = 10'0000'0007, g = 5; 不可以
ll qpow(ll x, ll n) {
        ll ret = 1;
        while(n) {
                if(n & 1) {
                        ret = ret * x % modulo;
                }
                x = x * x % modulo;
                n >>= 1;
        }
        return ret;
}
vector<ll> multiply(const vector<ll> &a, const vector<ll> &b) {
        int n = 1;
        while(n < a.size() + b.size()) {
                n <<= 1;
        }
        vector<ll> ca(n), cb(n);
        for(int i = 0; i < a.size(); ++i) {
                ca[i] = a[i];
        }
        for(int i = 0; i < b.size(); ++i) {
                cb[i] = b[i];
        }
        auto ntt = [](auto &&ntt, vector<ll> &f, bool invert) -> void {
                int n = f.size();
                if(n == 1) return;
                vector<ll> f0(n / 2), f1(n / 2);
                for(int i = 0; i < n / 2; ++i) {
                        f0[i] = f[2 * i];
                        f1[i] = f[2 * i + 1];
                }
                ntt(ntt, f0, invert);
                ntt(ntt, f1, invert);
                ll w = 1, wn = qpow(g, (modulo - 1) / n);
                if(invert) {
                        wn = qpow(wn, modulo - 2);
                }
                for(int t = 0; t < n / 2; ++t) {
                        ll u = f0[t], v = w * f1[t] % modulo;
                        f[t] = (u + v) % modulo;
                        f[t + n / 2] = (u - v + modulo) % modulo;
                        w = w * wn % modulo;
                }
        };
        ntt(ntt, ca, false);
        ntt(ntt, cb, false);
        for(int i = 0; i < n; ++i) {
                ca[i] = ca[i] * cb[i] % modulo;
        }
        ntt(ntt, ca, true);
        for(int i = 0; i < n; ++i) {
                ca[i] = ca[i] * qpow(n, modulo - 2) % modulo;
        }
```

```
        while(ca.size() > (a.size() + b.size() - 1)) {
                ca.pop_back();
        }
        return ca;
}
```

线性基

```
struct LinearBasis_XOR {
        LinearBasis_XOR() : base(61) {}
        void insert(ll val) {
                for(int i = 60; i >= 0; --i) {
                        if(((val >> i) & 1) == 0) continue;
                        if(base[i] == 0) {
                                base[i] = val;
                                return;
                        }
                        val ^= base[i];
                }
        }
        ll query_max() const {
                ll ans = 0;
                for(int i = 60; i >= 0; --i) {
                        if((ans ^ base[i]) > ans) {
                                ans ^= base[i];
                        }
                }
                return ans;
        }
private:
        vector<ll> base;
};
```

矩阵乘法和快速幂

```
template<class T> vector<vector<T>> matmul(const vector<vector<T>> &lhs, const vector<vector<T>> &rhs) {
        int M = lhs.size(), N = lhs[0].size(), P = rhs[0].size();
        vector<vector<T>> ret(M, vector<T>(P, 0));
        for(int i = 0; i < M; ++i) {
                for(int j = 0; j < P; ++j) {
                        for(int k = 0; k < N; ++k) {
                                ret[i][j] = ret[i][j] + lhs[i][k] * rhs[k][j];
                        }
                }
        }
        return ret;
}
template<class T> vector<vector<T>> matpow(vector<vector<T>> mat, ll N) {
        int M = mat.size();
        vector<vector<T>> ret(M, vector<T>(M, 0));
        for(int i = 0; i < M; ++i) {
                ret[i][i] = static_cast<T>(1);
        }
        while(N) {
                if(N & 1ll) {
                        ret = matmul(ret, mat);
                }
                mat = matmul(mat, mat);
                N >>= 1;
        }
        return ret;
}
```

高斯消元

```
vector<vector<ll>> gauss(const vector<vector<ll>> &a, const vector<vector<ll>> &b) {
    int r = a.size(), n = a[0].size(), m = b[0].size(), row = 0;
    vector<vector<ll>> aug(r, vector<ll>(n + m));
    for(int i = 0; i < r; ++i) {
        for(int j = 0; j < m + n; ++j) {
            aug[i][j] = (j < n) ? a[i][j] : b[i][j - n];
        }
    }
    vector<int> where(n, -1);
    for(int col = 0; col < n && row < r; ++col) {
        int sel = row;
        while(sel < r && aug[sel][col] == 0) ++sel;
        if(sel == r) continue;
        swap(aug[row], aug[sel]);
        ll inv = qpow(aug[row][col], modulo - 2);
        for(int j = col; j < n + m; ++j) {
            aug[row][j] = aug[row][j] * inv % modulo;
        }
        for(int i = 0; i < r; ++i) {
            if(i != row && aug[i][col]) {
                ll f = aug[i][col];
                for(int j = col; j < n + m; ++j) {
                    aug[i][j] = (aug[i][j] - f * aug[row][j] % modulo + modulo) % modulo;
                }
            }
        }
        where[col] = row;
        ++row;
    }
    for(int i = row; i < r; ++i) {
        bool flag = true;
        for(int j = 0; j < n; ++j) {
            if(aug[i][j] != 0) {
                flag = false;
                break;
            }
        }
        if(flag) {
            for(int j = 0; j < m; ++j) {
                if(aug[i][n + j]) throw runtime_error("No solution");
            }
        }
    }
    for(int col = 0; col < n; ++col) {
        if(where[col] == -1) throw runtime_error("Multiple solutions");
    }
    vector<vector<ll>> ret(n, vector<ll>(m, 0));
    for(int col = 0; col < n; ++col) {
        for(int j = 0; j < m; ++j) {
            ret[col][j] = aug[where[col]][n + j];
        }
    }
    return ret;
}
```

构造增广状态转移矩阵

问题	转移矩阵	初始值	结果
$\sum_{k=0}^{n}(aB^k)_{i,j}$	$T=\begin{bmatrix} B & 0 \\ e_j\cdot e_i^T & 1 \end{bmatrix}$	$z_0=\begin{bmatrix} a \\ a_i \end{bmatrix}$	$z_n=T^nz_0$ 的**最后一个分量**
$\sum_{k=0}^{n}\langle u,B^kv\rangle$	$T=\begin{bmatrix} B & 0 \\ u^T & 1 \end{bmatrix}$	$z_0=\begin{bmatrix} v \\ \langle u,v\rangle \end{bmatrix}$	$s_n=T^nz_0[back]$
$\sum_{i=0}^{n}A^i$	$T=\begin{bmatrix} A & I \\ 0 & I \end{bmatrix}$	无	T^{n+1} 的**右上角分块**

Lehmer码

`l[i] = k`代表后面有k个数比`a[i]`小

```
// 均为0-based排列
vector<int> lehmer(const vector<int> &a) {
        int n = a.size();
        vector<int> l(n);
        Fenwick bit(n);
        for(int i = 1; i <= n; ++i) {
                bit.update(i, 1);
        }
        for(int i = 0; i < n; ++i) {
                int x = a[i] + 1;
                l[i] = bit.query(n) - bit.query(x);
                bit.update(x, -1);
        }
        return l;
}
vector<int> rev_lehmer(const vector<int> &l) {
        int n = l.size();
        VST vst(n); // 使用权值线段树实现会快一点
        for(int i = 0; i < n; ++i) {
                vst.insert(i);
        }
        vector<int> ret(n);
        for(int i = 0; i < n; ++i) {
                ret[i] = vst.qvr(l[i] + 1);
                vst.erase(ret[i]);
        }
        return ret;
}
```

计算几何

常量与数据类型

```
using ld = long double;
constexpr ld eps = 1e-9;
int sign(ld a) {
        return (a < -eps) ? -1 : (a > eps) ? 1 : 0;
}
int cmp(ld a, ld b) {
        return sign(a - b);
}
constexpr ld pi = 3.14159265358979323846261;
constexpr ld inf = 1e12l;
```

点和向量

```
struct Point {
	ld x, y;
	Point() : x(0.0l), y(0.0l) {}
	Point(ld _x, ld _y) : x(_x), y(_y) {}
	Point(const complex<ld> &cd) : x(cd.real()), y(cd.imag()) {}
	operator complex<ld>() const {
		return complex<ld>(x, y);
	}
	Point operator+(const Point &p) const {
		return Point(x + p.x, y + p.y);
	}
	Point operator-(const Point &p) const {
		return Point(x - p.x, y - p.y);
	}
	Point operator*(ld z) const {
		return Point(z * x, z * y);
	}
	friend Point operator*(ld z, const Point &p) {
		return p * z;
	}
	Point operator/(ld z) const {
		return Point(x / z, y / z);
	}
	bool operator==(const Point &p) const {
		return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;
	}
	ld len() const {
		return sqrt(x * x + y * y);
	}
	// [0, 2pi)
	ld arg() const {
		ld ret = atan2(y, x);
		int c = cmp(ret, 0);
		return c == 1 ? ret : c == 0 ? 0.0l : ret + 2 * pi;
	}
	Point rotate(ld a) const {
		return Point(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
	}
};
using Vector = Point;
ld dot(const Vector &x, const Vector &y) {
	return x.x * y.x + x.y * y.y;
}
ld cross(const Vector &x, const Vector &y) {
	return x.x * y.y - x.y * y.x;
}
ld cross(const Point &o, const Point &a, const Point &b) {
	return cross(a - o, b - o);
}
bool argcmp(const Point &x, const Point &y) {
	bool bx = sign(x.y) == 1 || (sign(x.y) == 0 && sign(x.x) == 1),
	     by = sign(y.y) == 1 || (sign(y.y) == 0 && sign(y.x) == 1);
	if(bx != by) return bx;
	return sign(cross(x, y)) == 0;
}
```

```
ld dist(const Point &x, const Point &y) {
        return (x - y).len();
}
int to_left(const Vector &a, const Vector &b) {
        return sign(cross(a, b));
}
int to_left(const Point &a, const Point &b, const Point &c) {
        return sign(cross(a, b, c));
}
```

直线

```
struct Line {
        Point p, v;
        Line() {}
        Line(const Point &_p, const Vector &_v) : p(_p), v(_v) {}
};
int to_left(const Line &ln, const Point &p) {
        return to_left(ln.v, p - ln.p);
}
bool parallel(const Line &l1, const Line &l2) {
        return cmp(cross(l1.v, l2.v), 0.0l) == 0;
}
int is_inter(const Line &l1, const Line &l2) {
        return parallel(l1, l2) ? 0 : 1;
}
Point inter(const Line &a, const Line &b) {
        if(parallel(a, b))
                throw invalid_argument("a and b are parallel");
        Vector v = a.v * (cross(b.v, a.p - b.p) / cross(a.v, b.v));
        return a.p + v;
}
ld dist(const Point &p, const Line &ln) {
        return abs(cross(ln.v, p - ln.p)) / ln.v.len();
}
ld dist(const Line &l1, const Line &l2) {
        if(!parallel(l1, l2)) {
                return 0;
        }
        return dist(l1.p, l2);
}
Point proj(const Point &p, const Line &ln) {
        Vector v = ln.v * (dot(ln.v, p - ln.p) / (dot(ln.v, ln.v)));
        return ln.p + v;
}
bool is_on(const Point &p, const Line &ln) {
        return cmp(cross(ln.v, ln.p - p), 0.0l) == 0;
}
```

线段

```
struct Lineseg {
	Point a, b;
	Lineseg() {}
	Lineseg(const Point &_a, const Point &_b) : a(_a), b(_b) {}
	ld len() const {
		return (b - a).len();
	}
};
int is_on(const Point &p, const Lineseg &ls) {
	if(p == ls.a || p == ls.b) {
		return 2;
	}
	return to_left(p - ls.a, p - ls.b) == 0 && cmp(dot(p - ls.a, p - ls.b), 0) == -1;
}
int is_inter(const Line &ln, const Lineseg &ls) {
	int a = to_left(ln, ls.a), b = to_left(ln, ls.b);
	if(a == 0 || b == 0) {
		return 2;
	}
	return a == b ? 0 : 1;
}
int is_inter(const Lineseg &l1, const Lineseg &l2) {
	if(is_on(l1.a, l2) || is_on(l1.b, l2) || is_on(l2.a, l1) || is_on(l2.b, l1)) {
		return 2;
	}
	Line ln1(l1.a, l1.b - l1.a), ln2(l2.a, l2.b - l2.a);
	return to_left(ln1, l2.a) * to_left(ln1, l2.b) == -1 && to_left(ln2, l1.a) * to_left(ln2, l1.b) == -1;
}
ld dist(const Point &p, const Lineseg &ls) {
	if(is_on(p, ls) != 0) {
		return 0.0l;
	}
	if(cmp(dot(p - ls.a, ls.b - ls.a), 0) == -1 || cmp(dot(p - ls.b, ls.a - ls.b), 0) == -1) {
		return min(dist(p, ls.a), dist(p, ls.b));
	}
	Line l(ls.a, ls.b - ls.a);
	return dist(p, l);
}
```

多边形

```
struct Polygon {
	vector<Point> pts;
	Polygon() {}
	Polygon(const vector<Point> &p) : pts(p) {}
	Polygon(const vector<Line> &l) {
		pts.reserve(l.size());
		for(int i = 0; i < l.size(); ++i) {
			pts.emplace_back(inter(l[i], l[(i + 1) % l.size()]));
		}
	}
	ld area() const {
		ld ret = 0.0l;
		for(int i = 0; i < pts.size(); ++i) {
			ret += cross(pts[i], pts[(i + 1) % pts.size()]);
		}
		return ret / 2.0l;
	}
	ld circ() const {
		ld ret = 0.0l;
		for(int i = 0; i < pts.size(); ++i) {
			ret += (pts[i] - pts[(i + 1) % pts.size()]).len();
		}
		return ret;
	}
};
```

圓

```
struct Circle {
	Point c;
	ld r;
	Circle() : r(0.0l) {}
	Circle(const Point &_c, ld _r) : c(_c), r(_r) {}
	ld area() const {
		return pi * r * r;
	}
	ld circ() const {
		return 2.0l * pi * r;
	}
};

vector<Point> inter(const Circle &c1, const Circle &c2) {
	ld dis = (c2.c - c1.c).len();
	ld r1 = c1.r, r2 = c2.r;
	if(abs(dis) <= eps) {
		if(r1 == r2) {
			return {Point{}};
		}
		return {};
	}
	if (dis > r1 + r2 + eps || dis < abs(r1 - r2) + eps) {
		return {};
	}
	ld cosa = ((r1 * r1 + dis * dis - r2 * r2) / (2 * r1 * dis));
	ld alp = acos(min(max(cosa, -1.0l), 1.0l));
	Point v = c2.c - c1.c;
	v = v / v.len() * c1.r;
	return {c1.c + v.rotate(alp), c1.c + v.rotate(-alp)};
}

bool inter(const Circle &c, const Line &l, pair<Point, Point> &ret) {
	ld d = dist(c.c, l);
	if(sign(d) == 0) {
		ret.first = l.v * (1.0l / l.v.len()) * c.r + c.c;
		ret.second = l.v * (-1.0l / l.v.len()) * c.r + c.c;
		return true;
	}
	if(cmp(c.r, d) != 1) return false;
	Point vec;
	if(to_left(l.v, c.c - l.p) == 1) {
		vec = Point{ l.v.y, -l.v.x };
	} else {
		vec = Point{ -l.v.y, l.v.x };
	}
	vec = vec * (1.0l / vec.len()) * c.r;
	ld cosa = d / c.r;
	ld a = acos(cosa);
	ret.first = vec.rotate(-a) + c.c;
	ret.second = vec.rotate(a) + c.c;
	return true;
}
```

平面凸包

```
vector<Point> andrew(vector<Point> points) {
        sort(points.begin(), points.end(), [](const Point &a, const Point &b) {
                return a.x < b.x || (a.x == b.x && a.y < b.y);
        });
        points.erase(unique(points.begin(), points.end()), points.end());
        int n = points.size();
        if(n <= 2) {
                return points;
        }
        vector<Point> stk;
        for(int i = 0; i < n; ++i) {
                while(stk.size() >= 2 && sign(cross(stk[stk.size() - 2], stk.back(), points[i])) <= 0) {
                        stk.pop_back();
                }
                stk.push_back(points[i]);
        }
        int t = stk.size();
        for(int i = n - 2; i >= 0; --i) {
                while(stk.size() > t && sign(cross(stk[stk.size() - 2], stk.back(), points[i])) <= 0) {
                        stk.pop_back();
                }
                stk.push_back(points[i]);
        }
        stk.pop_back();
        return stk;
}
```

凸多边形

```
struct Convex : public Polygon {
	using Polygon::Polygon;
};

bool is_in(const Point &pt, const Convex &convex) {
	if(convex.size() == 1) {
		return pt == convex.pts[0];
	}
	if(convex.size() == 2) {
		return is_on(pt, Lineseg(convex.pts[0], convex.pts[1]));
	}
	auto check = [](const Point &a, const Point &b, const Point &c, const Point &p) {
		ld c1 = cross(b - a, p - a), c2 = cross(c - b, p - b), c3 = cross(a - c, p - c);
		return (sign(c1) != -1 && sign(c2) != -1 && sign(c3) != -1)
			|| (sign(c1) != 1 && sign(c2) != 1 && sign(c3) != 1);
	};
	int n = convex.size();
	Point pivot = convex.pts[0];
	if((sign(cross(convex.pts[1] - pivot, pt - pivot)) == -1)
	|| (sign(cross(convex.pts[n - 1] - pivot, pt - pivot)) == 1)) {
		return false;
	}
	int l = 1, r = n - 1;
	while(l + 1 < r) {
		int mid = (l + r) >> 1;
		if(sign(cross(convex.pts[mid] - pivot, pt - pivot)) != -1) {
			l = mid;
		} else {
			r = mid;
		}
	}
	int nxt = (l == n - 1) ? 1 : l + 1;
	return check(pivot, convex.pts[l], convex.pts[nxt], pt);
}

pair<Point, Point> tangent(const Point &pt, const Convex &convex) {
	if(is_in(pt, convex)) throw runtime_error("pt is in the convex");
	int n = convex.size();
	auto peak = [&](int l, int r, bool find_r) {
		while(l < r - 1) {
			int mid = (l + r) / 2;
			if(find_r ^ (to_left(convex.pts[(mid + n - 1) % n] - pt, convex.pts[mid] - pt) == 1)) {
				r = mid;
			} else {
				l = mid;
			}
		}
		return l;
	};
	if(to_left(convex.pts[0] - pt, convex.pts[1] - pt) == 0) {
		int idx = peak(2, n, cmp(dist(convex.pts[0], pt), dist(convex.pts[1], pt)) == -1);
		return { convex.pts[0], convex.pts[idx] };
	}
	bool all_left = true, all_right = true;
	auto fun = [&](int x) {
```

```
                if(x == 1) all_right = false;
                if(x == -1) all_left = false;
        };
        fun(to_left(convex.pts[0] - pt, convex.pts[1] - pt));
        fun(to_left(convex.pts[0] - pt, convex.pts[n - 1] - pt));
        if(all_left || all_right) {
                int idx = peak(1, n, all_left);
                return { convex.pts[0], convex.pts[idx] };
        }
        int l = 1, r = n;
        while(l < r - 1) {
                int mid = (l + r) / 2;
                if(to_left(convex.pts[0] - pt, convex.pts[mid] - pt) == 1) {
                        l = mid;
                } else {
                        r = mid;
                }
        }
        int split = l;
        bool flag = (to_left(convex.pts[0] - pt, convex.pts[1] - pt) == 1);
        int i1 = peak(0, split + 1, flag), i2 = peak(split, n, !flag);
        return { convex.pts[i1], convex.pts[i2] };
}
```

半平面交

```
vector<Line> half_inter(vector<Line> lines) {
	lines.push_back({{-inf, 0.0l}, {0.0l, -1.0l}});
	lines.push_back({{inf, 0.0l}, {0.0l, 1.0l}});
	lines.push_back({{0.0l, inf}, {-1.0l, 0.0l}});
	lines.push_back({{0.0l, -inf}, {1.0l, 0.0l}});
	sort(lines.begin(), lines.end(), [](const Line &a, const Line &b) {
		int c = cmp(a.v.arg(), b.v.arg());
		if(c != 0)
			return c == -1;
		return sign(cross(a.v, b.p - a.p)) < 0;
	});
	deque<Line> dq;
	auto bad = [](const Line &l, const Line &b, const Line &c) {
		try {
			auto p = inter(b, c);
			return to_left(l, p) < 0;
		} catch(const invalid_argument &) {
			return false;
		}
	};
	for(auto &l : lines) {
		if(!dq.empty() && cmp(l.v.arg(), dq.back().v.arg()) == 0) {
			if(to_left(l, dq.back().p + dq.back().v) < 0)
				dq.back() = l;
			continue;
		}
		while(dq.size() > 1 && bad(l, dq.back(), dq[dq.size() - 2]))
			dq.pop_back();
		while(dq.size() > 1 && bad(l, dq[0], dq[1]))
			dq.pop_front();
		dq.push_back(l);
	}
	while(dq.size() > 1 && bad(dq[0], dq.back(), dq[dq.size() - 2]))
		dq.pop_back();
	while(dq.size() > 1 && bad(dq.back(), dq[0], dq[1]))
		dq.pop_front();
	vector<Line> ret(dq.begin(), dq.end());
	int m = ret.size();
	if(m < 3)
		return {};
	vector<Point> poly;
	poly.reserve(m);
	for(int i = 0; i < m; ++i) {
		try {
			poly.emplace_back(inter(ret[i], ret[(i + 1) % m]));
		} catch(const invalid_argument &) {
			return {};
		}
	}
	if(poly.size() != m)
		return {};
	return ret;
}
```

扫描线

```
struct Rectangle {
	Point p1, p2;
};

struct Event {
	int delta, l, r;
	ld y;
};

struct SegTree {
	SegTree(const vector<ld> &_xs) : nodes(4 * _xs.size()), xs(_xs) {
		int n = xs.size();
		_build(0, 0, n);
	}
	ld query() const {
		return nodes[0].len;
	}
	// 注意左闭右开
	void update(int l, int r, int v) {
		_update(0, l, r, v);
	}
private:
	struct Node {
		int l, r, cnt;
		ld len;
	};
	vector<Node> nodes;
	vector<ld> xs;
	void _pushup(int u) {
		if(nodes[u].cnt > 0) {
			nodes[u].len = xs[nodes[u].r] - xs[nodes[u].l];
		} else if(nodes[u].r - nodes[u].l == 1) {
			nodes[u].len = 0.0l;
		} else {
			int lson = (u << 1) + 1, rson = (u << 1) + 2;
			nodes[u].len = nodes[lson].len + nodes[rson].len;
		}
	}
	void _build(int u, int l, int r) {
		nodes[u] = { l, r, 0, 0.0l };
		if(r - l > 1) {
			int mid = (l + r) >> 1, lson = (u << 1) + 1, rson = (u << 1) + 2;
			_build(lson, l, mid);
			_build(rson, mid, r);
		}
	}
	void _update(int u, int l, int r, int v) {
		if(nodes[u].l >= r || nodes[u].r <= l) return;
		if(nodes[u].l >= l && nodes[u].r <= r) {
			nodes[u].cnt += v;
			_pushup(u);
			return;
		}
		int lson = (u << 1) + 1, rson = (u << 1) + 2;
		_update(lson, l, r, v);
```

```
                _update(rson, l, r, v);
                _pushup(u);
        }
};

ld scanline(const vector<Rectangle> &rects) {
        int n = rects.size();
        vector<ld> xs(2 * n);
        vector<Event> events(2 * n);
        for(int i = 0; i < n; ++i) {
                xs[2 * i] = rects[i].p1.x;
                xs[2 * i + 1] = rects[i].p2.x;
        }
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        auto getid = [&](ld x) -> int {
                return lower_bound(xs.begin(), xs.end(), x) - xs.begin();
        };
        for(int i = 0; i < n; ++i) {
                events[2 * i] = { 1, getid(rects[i].p1.x), getid(rects[i].p2.x), rects[i].p1.y };
                events[2 * i + 1] = { -1, getid(rects[i].p1.x), getid(rects[i].p2.x), rects[i].p2.y };
        }
        sort(events.begin(), events.end(), [](const Event &a, const Event &b) {
                return cmp(a.y, b.y) == -1;
        });
        SegTree seg(xs);
        ld lasty = events[0].y;
        ld ans = 0.0l;
        for(auto &e : events) {
                ans += seg.query() * (e.y - lasty);
                seg.update(e.l, e.r, e.delta);
                lasty = e.y;
        }
        return ans;
}
```

旋转卡壳

```
ld farthest_dist(const vector<Point> &pts) {
    int n = pts.size();
    auto hull = andrew(pts);
        int m = hull.size();
        int j = 1;
        ld ans = 0;
        for(int i = 0; i < m; ++i) {
                int u = (i + 1) % m;
                while(cross(hull[i], hull[u], hull[(j + 1) % m]) > cross(hull[i], hull[u], hull[j])) {
                        j = (j + 1) % m;
                }
                ans = max({ ans, (hull[i] - hull[j]).len2(), (hull[u] - hull[j]).len2() });
        }
    return ans;
}
```

字符串

KMP

```
// 得到用于KMP的数组
vector<int> kmp_f(const string &s) {
        int n = s.size();
        vector<int> f(n, 0);
        for(int i = 1, j = 0; i < n; ++i) {
                while(j > 0 && s[i] != s[j]) {
                        j = f[j - 1];
                }
                if(s[i] == s[j]) {
                        ++j;
                }
                f[i] = j;
        }
        return f;
}
// 寻找第一次在off后面haystack中的needle
int kmp_find(const string &haystack, const string &needle, int off = 0) {
        int n = needle.size(), m = haystack.size();
        vector<int> f = kmp_f(needle);
        for(int i = 0, j = off; j < m;) {
                if(haystack[j] == needle[i]) {
                        ++i;
                        ++j;
                        if(i == n) {
                                return j - n;
                        }
                } else {
                        if(i == 0) {
                                ++j;
                        } else {
                                i = f[i - 1];
                        }
                }
        }
        return -1;
}
// 返回haystack中有多少个needle，可重（即10101有两个101）
int kmp_count(const string &haystack, const string &needle) {
        int n = needle.size(), m = haystack.size();
        int ret = 0;
        vector<int> f = kmp_f(needle);
        for(int i = 0, j = 0; j < m;) {
                if(haystack[j] == needle[i]) {
                        ++i;
                        ++j;
                        if(i == n) {
                                ++ret;
                                i = f[i - 1];
                        }
                } else {
                        if(i == 0) {
                                ++j;
```

```
                        } else {
                                i = f[i - 1];
                        }
                }
        }
        return ret;
}
```

Z函数（扩展KMP）

```
vector<int> zfn(const string &s) {
    int n = s.size();
    vector<int> z(n);
    for(int i = 1, l = 0, r = 0; i < n; ++i) {
        if(i <= r && z[i - l] < r - i + 1) {
            z[i] = z[i - l];
        } else {
            z[i] = max(0, r - i + 1);
            while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
                ++z[i];
            }
        }
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
```

字典树

```
struct Trie {
        Trie() : nodes(1) {}
        void insert(const string &str) {
                int rt = 0;
                for(int i = 0; i < str.size(); ++i) {
                        if((nodes[rt].nxt[getnum(str[i])]) == -1) {
                                nodes[rt].nxt[getnum(str[i])] = nodes.size();
                                nodes.emplace_back();
                        }
                        nodes[rt].cnt++;
                        rt = nodes[rt].nxt[getnum(str[i])];
                }
                nodes[rt].cnt++;
                nodes[rt].end = true;
        }
        bool find(const string &str) const {
                int rt = 0;
                for(int i = 0; i < str.size(); ++i) {
                        if((nodes[rt].nxt[getnum(str[i])]) == -1) {
                                return false;
                        }
                        rt = nodes[rt].nxt[getnum(str[i])];
                }
                return nodes[rt].end;
        }
        int find_prefix(const string &str) const {
                int rt = 0;
                for(int i = 0; i < str.size(); ++i) {
                        if((nodes[rt].nxt[getnum(str[i])]) == -1) {
                                return 0;
                        }
                        rt = nodes[rt].nxt[getnum(str[i])];
                }
                return nodes[rt].cnt;
        }
private:
        struct Node {
                array<int, 65> nxt;
                bool end;
                int cnt;
                Node() : end(false), cnt(0) {
                        nxt.fill(-1);
                }
        };
        vector<Node> nodes;
        static int getnum(char x) {
                if(x >= 'A' && x <= 'Z') {
                        return x - 'A';
                } else if(x >= 'a' && x <= 'z') {
                        return x - 'a' + 26;
                } else {
                        return x - '0' + 52;
                }
```

```
        }
};
```

Manacher

```
string manacher(const string &_s) {
        string s = "$";
        for(char ch : _s) {
                s += ch;
                s += '$';
        }
        int n = s.size();
        int max_right = 0;
        int center = 0;
        vector<int> dp(n, 0);
        auto expand = [&](int left, int right) {
                while(left >= 0 && right < n && s[left] == s[right]) {
                        --left;
                        ++right;
                }
                return (right - left) / 2;
        };
        for(int i = 0; i < n; ++i) {
                int mirror = 2 * center - i;
                if(i >= max_right) {
                        dp[i] = expand(i, i);
                        max_right = i + dp[i];
                        center = i;
                } else if(dp[mirror] == max_right - i) {
                        dp[i] = expand(i - dp[mirror], i + dp[mirror]);
                        max_right = i + dp[i];
                        center = i;
                } else {
                        dp[i] = min(dp[mirror], max_right - i);
                }
        }
        auto it = max_element(dp.begin(), dp.end());
        int id = it - dp.begin(), len = *it;
        string ret;
        for(int i = id - len + 1; i < id + len; ++i) {
                if(s[i] != '$') {
                        ret += s[i];
                }
        }
        return ret;
}
```

AC自动机

千万不要忘了build!

```
struct ACAM {
        ACAM() : nodes(1) {}
        void insert(const string &str) {
                int now = 0;
                for(char ch : str) {
                        int id = ch - 'a';
                        int v = nodes[now].nxt1[id];
                        if(v == -1) {
                                v = nodes[now].nxt1[id] = nodes[now].nxt2[id] = nodes.size();
                                nodes.emplace_back();
                        }
                        now = v;
                }
                nodes[now].cnt++;
        }
        // 不优化的，对于某些特定的题需要
        // void build() {
        //      queue<int> q;
        //      nodes[0].fail = 0;
        //      for(int i = 0; i < 26; ++i) {
        //              if(nodes[0].nxt[i] != -1) {
        //                      int v = nodes[0].nxt[i];
        //                      nodes[v].fail = 0;
        //                      q.push(v);
        //              }
        //      }
        //      while(!q.empty()) {
        //              int u = q.front();
        //              q.pop();
        //              for(int i = 0; i < 26; ++i) {
        //                      int v = nodes[u].nxt[i];
        //                      if(v == -1) {
        //                              continue;
        //                      }
        //                      int f = nodes[u].fail;
        //                      while(f != 0 && nodes[f].nxt[i] == -1) {
        //                              f = nodes[f].fail;
        //                      }
        //                      nodes[v].fail = max(0, nodes[f].nxt[i]);
        //                      q.push(v);
        //              }
        //      }
        // }
        // int query(const string &str) const {
        //      vector<int> cnter(nodes.size(), 0);
        //      int now = 0;
        //      for(char ch : str) {
        //              int id = ch - 'a';
        //              while(now != 0 && nodes[now].nxt[id] == -1) {
        //                      now = nodes[now].fail;
        //              }
        //              now = max(nodes[now].nxt[id], 0);
        //              int v = now;
        //              while(v != 0) {
        //                      cnter[v] = nodes[v].cnt;
        //                      v = nodes[v].fail;
        //              }
```

```
//      }
//      int ans = 0;
//      for(int i : cnter) {
//              ans += i;
//      }
//      return ans;
// }
void build() {
        queue<int> q;
        nodes[0].fail = 0;
        for(int i = 0; i < 26; ++i) {
                int v = nodes[0].nxt2[i];
                if(v == -1) {
                        nodes[0].nxt2[i] = 0;
                } else {
                        nodes[v].fail = 0;
                        q.push(v);
                }
        }
        while(!q.empty()) {
                int now = q.front();
                q.pop();
                for(int i = 0; i < 26; ++i) {
                        int v = nodes[now].nxt2[i];
                        if(v == -1) {
                                nodes[now].nxt2[i] = nodes[nodes[now].fail].nxt2[i];
                        } else {
                                nodes[v].fail = nodes[nodes[now].fail].nxt2[i];
                                q.push(v);
                        }
                }
        }
        q.push(0);
        while(!q.empty()) {
                int u = q.front();
                q.pop();
                for(int i = 0; i < 26; ++i) {
                        int v = nodes[u].nxt1[i];
                        if(v == -1) {
                                continue;
                        }
                        q.push(v);
                        int fu = nodes[u].fail;
                        if(fu != 0 && nodes[fu].cnt == 0) {
                                nodes[u].fail = nodes[fu].fail;
                        }
                }
        }
}
int find(const string &s) const {
        int now = 0;
        for(char ch : s) {
                int v = nodes[now].nxt1[ch - 'a'];
                if(v == -1) {
                        return -1;
                }
                now = v;
```

```
                }
                return now;
        }
        vector<int> query(const string &str) const {
                int now = 0;
                vector<int> ret(nodes.size(), 0);
                for(char ch : str) {
                        int id = ch - 'a';
                        now = nodes[now].nxt2[id];
                        int u = now;
                        while(u != 0) {
                                ret[u]++;
                                u = nodes[u].fail;
                        }
                }
                return ret;
        }
        int size() const {
                return nodes.size();
        }
private:
        struct Node {
                int cnt;
                int fail;
                array<int, 26> nxt1, nxt2;
                Node() : cnt(0), fail(0) {
                        nxt1.fill(-1);
                        nxt2.fill(-1);
                }
        };
        vector<Node> nodes;
};
```

后缀数组

```
vector<int> suffix_array(const string &str) {
        int n = str.size();
        int m = 128;
        vector<int> rk(n * 3, -1), sa(n), id(n), cnt(max(n, m), 0);
        int p = 0;
        for(int i = 0; i < n; ++i) {
                rk[i] = str[i];
                cnt[rk[i]]++;
        }
        for(int i = 1; i < m; ++i) {
                cnt[i] += cnt[i - 1];
        }
        for(int i = n - 1; i >= 0; --i) {
                cnt[rk[i]]--;
                sa[cnt[rk[i]]] = i;
        }
        for(int w = 1; ; w <<= 1, m = p + 1) {
                int cur = 0;
                for(int i = n - w; i < n; ++i) {
                        id[cur] = i;
                        ++cur;
                }
                for(int i = 0; i < n; ++i) {
                        if(sa[i] >= w) {
                                id[cur] = sa[i] - w;
                                ++cur;
                        }
                }
                cnt.assign(max(n, m), 0);
                for(int i = 0; i < n; ++i) {
                        cnt[rk[i]]++;
                }
                for(int i = 1; i < m; ++i) {
                        cnt[i] += cnt[i - 1];
                }
                for(int i = n - 1; i >= 0; --i) {
                        cnt[rk[id[i]]]--;
                        sa[cnt[rk[id[i]]]] = id[i];
                }
                p = 0;
                vector<int> oldrk = rk;
                rk[sa[0]] = 0;
                for(int i = 1; i < n; ++i) {
                        if(oldrk[sa[i]] != oldrk[sa[i - 1]] || oldrk[sa[i] + w] != oldrk[sa[i - 1] + w]) {
                                ++p;
                        }
                        rk[sa[i]] = p;
                }
                if(p == n - 1) {
                        break;
                }
        }
```

```
        return sa;
}
```

Height数组

```
vector<int> height(const string &str) {
        int n = str.size();
        vector<int> sa = suffix_array(str), rk(n), h(n);
        for(int i = 0; i < n; ++i) {
                rk[sa[i]] = i;
        }
        for(int i = 0, k = 0; i < n; ++i) {
                if(rk[i] == 0) {
                        continue;
                }
                if(k > 0) {
                        --k;
                }
                while(str[i + k] == str[sa[rk[i] - 1] + k]) {
                        ++k;
                }
                h[rk[i]] = k;
        }
        return h;
}
```

后缀自动机

```
struct SAM {
	SAM() : nodes(1, Node(-1, 0)), last(0) {}
	void insert(char ch) {
		const int id = ch - 'a';
		const int cur = nodes.size();
		nodes.emplace_back(-1, nodes[last].len + 1);
		int p = last;
		while(p != -1 && nodes[p].nxt[id] == -1) {
			nodes[p].nxt[id] = cur;
			p = nodes[p].link;
		}
		if(p == -1) {
			nodes[cur].link = 0;
		} else {
			int q = nodes[p].nxt[id];
			if(nodes[p].len + 1 == nodes[q].len) {
				nodes[cur].link = q;
			} else {
				int clone = nodes.size();
				nodes.emplace_back(nodes[q].link, nodes[p].len + 1);
				nodes[clone].nxt = nodes[q].nxt;
				while(p != -1 && nodes[p].nxt[id] == q) {
					nodes[p].nxt[id] = clone;
					p = nodes[p].link;
				}
				nodes[q].link = nodes[cur].link = clone;
			}
		}
		last = cur;
		ends.push_back(cur);
	}
	// 洛谷P3804 求出 S 的所有出现次数不为 1 的子串的出现次数乘上该子串长度的最大值。
	ll solve() const {
		int n = nodes.size();
		vector<int> indeg(n, 0), topoorder;
		topoorder.reserve(n);
		for(int i = 0; i < n; ++i) {
			if(nodes[i].link != -1) {
				indeg[nodes[i].link]++;
			}
		}
		queue<int> q;
		for(int i = 0; i < n; ++i) {
			if(indeg[i] == 0) {
				q.push(i);
			}
		}
		while(!q.empty()) {
			int u = q.front();
			q.pop();
			topoorder.push_back(u);
			if(nodes[u].link != -1) {
				indeg[nodes[u].link]--;
				if(indeg[nodes[u].link] == 0) {
					q.push(nodes[u].link);
```

```
                                }
                        }
                }
                ll ans = 0;
                vector<ll> occur(n, 0);
                for(int e : ends) {
                        occur[e] = 1;
                }
                for(int i : topoorder) {
                        if(nodes[i].link != -1) {
                                occur[nodes[i].link] += occur[i];
                        }
                        if(occur[i] != 1) {
                                ans = max(ans, occur[i] * nodes[i].len);
                        }
                }
                return ans;
        }
private:
        struct Node {
                array<int, 26> nxt;
                int link;
                int len;
                Node(int lk, int ln) : link(lk), len(ln) {
                        nxt.fill(-1);
                }
        };
        vector<Node> nodes;
        vector<int> ends;
        int last;
};
```

序列自动机

```
struct SeqAM {
	explicit SeqAM(const string &s) : n(s.size()), nxt(s.size() + 2, [&] {
		array<int, 26> ret;
		ret.fill(s.size() + 1);
		return ret;
	}()) {
		for(int i = n - 1; i >= 0; --i) {
			nxt[i] = nxt[i + 1];
			nxt[i][s[i] - 'a'] = i + 1;
		}
	}
	bool match(const string &t) const {
		int now = 0;
		for(char c : t) {
			now = nxt[now][c - 'a'];
		}
		return now != (n + 1);
	}
private:
	int n;
	vector<array<int, 26>> nxt;
};
```

回文自动机

```
struct PAM {
	PAM() : s("#"), last(1) {
		nodes.emplace_back(-1, 0);
		nodes.emplace_back(0, 0);
	}
	int insert(char ch) {
		s += ch;
		int pos = s.size() - 1;
		int p = last, id = ch - 'a';
		while(s[pos - nodes[p].len - 1] != ch) {
			p = nodes[p].fail;
		}
		if(nodes[p].nxt[id] == -1) {
			int cur = nodes.size();
			nodes[p].nxt[id] = cur;
			nodes.emplace_back(nodes[p].len + 2, 0);
			if(nodes[cur].len == 1) {
				nodes[cur].fail = 1;
			} else {
				int f = nodes[p].fail;
				while(s[pos - nodes[f].len - 1] != ch) {
					f = nodes[f].fail;
				}
				nodes[cur].fail = nodes[f].nxt[id];
			}
			if(nodes[cur].len == 1) {
				nodes[cur].cnt = 1;
			} else {
				nodes[cur].cnt = nodes[nodes[cur].fail].cnt + 1;
			}
		}
		last = nodes[p].nxt[id];
		return nodes[last].cnt;
	}
private:
	struct Node {
		int len, fail, cnt;
		array<int, 26> nxt;
		Node(int l, int f) : len(l), fail(f), cnt(0) {
			nxt.fill(-1);
		}
	};
	vector<Node> nodes;
	string s;
	int last;
};
```

字符串哈希

```
struct StringHash {
        explicit StringHash(const string &s) : p1(s.size() + 1, 0), p2(s.size() + 1, 0) {
                for(int i = 0; i < s.size(); ++i) {
                        p1[i + 1] = (p1[i] * mul1 + s[i]) % modulo;
                        p2[i + 1] = p2[i] * mul2 + s[i];
                }
        }
        ull substr(int l, int r) const {
                ull ret1 = (p1[r] - p1[l] * pmul1[r - l] % modulo + modulo) % modulo;
                ull ret2 = p2[r] - p2[l] * pmul2[r - l];
                return (ret1 << 3) ^ (ret1 >> 5) ^ ret2;
        }
private:
        vector<ull> p1, p2;
        static inline const ull c = (ull)chrono::steady_clock::now().time_since_epoch().count();
        static inline const ull mul1 = c % 131 + 131, mul2 = c % 13331 + 13331;
        static inline ull pmul1[maxn], pmul2[maxn];
        static inline int init = [&] {
                pmul1[0] = pmul2[0] = 1;
                for(int i = 1; i < maxn; ++i) {
                        pmul1[i] = pmul1[i - 1] * mul1 % modulo;
                        pmul2[i] = pmul2[i - 1] * mul2;
                }
                return 0;
        }();
};
```

Lyndon分解

```
vector<string> duval(const string &s) {
    int n = s.size();
    vector<string> ret;
    for(int i = 0; i < n;) {
        int j = i + 1, k = i;
        while(j < n && s[k] <= s[j]) {
            (s[k] < s[j]) ? (k = i) : (k++);
            ++j;
        }
        while(i <= k) {
            ret.push_back(s.substr(i, j - k));
            i += (j - k);
        }
    }
    return ret;
}
```

最小表示法

```
string minstr(const string &str) {
        int n = str.size(), i = 0, j = 1, k = 0;
        string s = str + str;
        while(i < n && j < n) {
                while(k < n && s[i + k] == s[j + k]) {
                        ++k;
                }
                if(k == n)
                        break;
                if(s[i + k] > s[j + k]) {
                        i += (k + 1);
                } else {
                        j += (k + 1);
                }
                if(i == j) {
                        ++j;
                }
                k = 0;
        }
        return s.substr(min(i, j), n);
}
```

贪心

一组交叉直线经过所有点

```
struct Point {
        int x, y;
};
bool cross_all(const vector<Point> &points) {
        bool okx = true, oky = true;
        int xb = -1, yb = -1;
        for(auto [x, y] : points) {
                if(x != points[0].x) {
                        if(yb == -1) {
                                yb = y;
                        } else if(yb != y) {
                                okx = false;
                        }
                }
                if(y != points[0].y) {
                        if(xb == -1) {
                                xb = x;
                        } else if(xb != x) {
                                oky = false;
                        }
                }
        }
        return okx || oky;
}
```

二分

最长上升子序列

```
vector<int> lis(const vector<int> &nums) {
        int n = nums.size();
        vector<int> dp, pos(n), pre(n, -1);
        for(int i = 0; i < n; ++i) {
                auto it = lower_bound(dp.begin(), dp.end(), nums[i]);
                if(it == dp.end()) dp.push_back(nums[i]);
                else *it = nums[i];
                int j = it - dp.begin();
                pos[j] = i;
                if(j != 0) pre[i] = pos[j - 1];
        }
        vector<int> ret;
        for(int i = pos[dp.size() - 1]; i != -1; i = pre[i]) {
                ret.push_back(nums[i]);
        }
        reverse(ret.begin(), ret.end());
        return ret;
}
```

二分答案

```
template<class T> T binary(auto &&check, T minv, T maxv, T eps, T delta) {
        T ans{}, hi = maxv, lo = minv;
        while((maxv - minv) > eps) {
                T mid = (minv + maxv) / 2;
                if(check(mid)) {
                        ans = mid;
                        hi = mid - delta;
                } else {
                        lo = mid + delta;
                }
        }
        return ans;
}
```

三分

```
int peak(const vector<int> &nums) {
        int n = nums.size();
        int l = 0, r = n - 1;
        int ll = (r - l) / 3, rr = (r - l) * 2 / 3;
        while(l < r) {
                if(nums[ll] < nums[rr]) {
                        l = ll + 1;
                } else {
                        r = rr;
                }
                ll = l + (r - l) / 3;
                rr = l + (r - l) * 2 / 3;
        }
        return l;
}
```

杂项

高精

```
struct HPINT {
	HPINT() : nums(1, 0), neg(false) {}
	HPINT(ll val) : neg(false) {
		if(val < 0) {
			neg = true;
			val = -val;
		}
		while(val) {
			int t = val % 10;
			nums.push_back(t);
			val /= 10;
		}
		if(nums.empty()) {
			nums = vector<int>(1, 0);
		}
	}
	HPINT &operator+=(const HPINT &r) {
		if(neg) {
			if(r.neg) {
				_add(nums, r.nums);
			} else {
				if(_cmp(nums, r.nums) == 1) {
					_sub(nums, r.nums);
				} else {
					vector<int> rnums = r.nums;
					_sub(rnums, nums);
					nums.swap(rnums);
					neg = false;
				}
			}
		} else {
			if(r.neg) {
				if(_cmp(nums, r.nums) != -1) {
					_sub(nums, r.nums);
				} else {
					vector<int> rnums = r.nums;
					_sub(rnums, nums);
					nums.swap(rnums);
					neg = true;
				}
			} else {
				_add(nums, r.nums);
			}
		}
		return *this;
	}
	HPINT &operator-=(const HPINT &r) {
		if(neg) {
			if(r.neg) {
				int c = _cmp(nums, r.nums);
				if(c == 1) {
					_sub(nums, r.nums);
```

```
                        } else {
                                vector<int> rnums = r.nums;
                                _sub(rnums, nums);
                                nums.swap(rnums);
                                neg = false;
                        }
                } else {
                        _add(nums, r.nums);
                }
        } else {
                if(r.neg) {
                        _add(nums, r.nums);
                } else {
                        int c = _cmp(nums, r.nums);
                        if(c == -1) {
                                vector<int> rnums = r.nums;
                                _sub(rnums, nums);
                                nums.swap(rnums);
                                neg = true;
                        } else {
                                _sub(nums, r.nums);
                        }
                }
        }
        return *this;
}
HPINT &operator*=(const HPINT &r) {
        neg ^= r.neg;
        _mul(nums, r.nums);
        return *this;
}
friend ostream &operator<<(ostream &os, const HPINT &hp) {
        if(hp.neg) {
                os << '-';
        }
        for(int i = hp.nums.size() - 1; i >= 0; --i) {
                os << hp.nums[i];
        }
        return os;
}
friend istream &operator>>(istream &is, HPINT &hp) {
        char ch = is.rdbuf()->sbumpc();
        while(ch < '0' || ch > '9') {
                if(ch == '-') {
                        hp.neg = true;
                }
                ch = is.rdbuf()->sbumpc();
        }
        string buffer;
        while(ch >= '0' && ch <= '9') {
                buffer += ch;
                ch = is.rdbuf()->sbumpc();
        }
        hp.nums.resize(buffer.size());
        int n = buffer.size();
        for(int i = 0; i < n; ++i) {
                hp.nums[i] = buffer[n - i - 1] - '0';
```

```
                }
                return is;
        }
        bool operator>(const HPINT &r) const {
                if(neg != r.neg) {
                        return !neg && r.neg;
                }
                int c = _cmp(nums, r.nums);
                if(neg) {
                        return c == -1;
                } else {
                        return c == 1;
                }
        }
        bool operator<(const HPINT &r) const {
                return r > (*this);
        }
        bool operator==(const HPINT &r) const {
                return neg == r.neg && _cmp(nums, r.nums) == 0;
        }
        HPINT operator+(const HPINT &r) const {
                HPINT ret = *this;
                ret += r;
                return ret;
        }
        HPINT operator-(const HPINT &r) const {
                HPINT ret = *this;
                ret -= r;
                return ret;
        }
        HPINT operator*(const HPINT &r) const {
                HPINT ret = *this;
                ret *= r;
                return ret;
        }
        bool operator>=(const HPINT &r) const {
                return !operator<(r);
        }
        bool operator<=(const HPINT &r) const {
                return !operator>(r);
        }
private:
        vector<int> nums;
        bool neg;
        static void _add(vector<int> &a, const vector<int> &b) {
                int carry = 0;
                for(int i = 0; i < b.size(); ++i) {
                        if(i >= a.size()) {
                                a.push_back(0);
                        }
                        a[i] += (b[i] + carry);
                        carry = 0;
                        if(a[i] >= 10) {
                                carry = 1;
                                a[i] -= 10;
                        }
                }
```

```
        int i = b.size();
        while(carry != 0) {
                if(i >= a.size()) {
                        a.push_back(0);
                }
                a[i] += carry;
                carry = 0;
                if(a[i] >= 10) {
                        carry = 1;
                        a[i] -= 10;
                }
                ++i;
        }
}
static void _sub(vector<int> &a, const vector<int> &b) {
        int borrow = 0;
        for(int i = 0; i < b.size(); ++i) {
                a[i] -= (borrow + b[i]);
                borrow = 0;
                if(a[i] < 0) {
                        a[i] += 10;
                        borrow = 1;
                }
        }
        int i = b.size();
        while(borrow && i < a.size()) {
                a[i] -= borrow;
                borrow = 0;
                if(a[i] < 0) {
                        a[i] += 10;
                        borrow = 1;
                }
                i++;
        }
        while(a.size() > 1 && a.back() == 0) {
                a.pop_back();
        }
}
static void _mul(vector<int> &a, const vector<int> &b) {
        using cd = complex<double>;
        constexpr double pi = 3.14159265358979323846264338327950288;
        int n = 1;
        while(n < (int)(a.size() + b.size())) {
                n <<= 1;
        }
        vector<cd> fa(n), fb(n);
        for(int i = 0; i < (int)a.size(); ++i) {
                fa[i] = a[i];
        }
        for(int i = 0; i < (int)b.size(); ++i) {
                fb[i] = b[i];
        }
        auto fft = [](auto &&fft, vector<cd> &f, bool invert) -> void {
                int n = f.size();
                if(n == 1)
                        return;
                vector<cd> f0(n / 2), f1(n / 2);
```

```
                        for(int i = 0; i < n / 2; ++i) {
                                f0[i] = f[2 * i];
                                f1[i] = f[2 * i + 1];
                        }
                        fft(fft, f0, invert);
                        fft(fft, f1, invert);
                        double theta = 2 * pi / n * (invert ? -1 : 1);
                        cd wt = 1, w(cos(theta), sin(theta));
                        for(int t = 0; t < n / 2; ++t) {
                                cd u = f0[t], v = wt * f1[t];
                                f[t] = u + v;
                                f[t + n / 2] = u - v;
                                wt *= w;
                        }
                };
                fft(fft, fa, false);
                fft(fft, fb, false);
                for(int i = 0; i < n; ++i) {
                        fa[i] *= fb[i];
                }
                fft(fft, fa, true);
                a.resize(n);
                int carry = 0;
                for(int i = 0; i < n; ++i) {
                        int num = int(fa[i].real() / n + 0.5) + carry;
                        carry = num / 10;
                        a[i] = num % 10;
                }
                while(carry) {
                        a.push_back(carry % 10);
                        carry /= 10;
                }
                while(a.size() > 1 && a.back() == 0) {
                        a.pop_back();
                }
        }
        // 1 gt, -1 lt, 0 eq
        static int _cmp(const vector<int> &a, const vector<int> &b) {
                if(a.size() > b.size())
                        return 1;
                if(a.size() < b.size())
                        return -1;
                for(int n = a.size(), i = n - 1; i >= 0; --i) {
                        if(a[i] > b[i])
                                return 1;
                        if(a[i] < b[i])
                                return -1;
                }
                return 0;
        }
};
```

快读

```
struct Qread {
	Qread() noexcept : state(true) {}
	template<integral T> Qread &operator>>(T &val) noexcept {
		if(!state) {
			val = 0;
			return *this;
		}
		T x = 0, f = 1;
		char ch = cin.rdbuf()->sbumpc();
		while(ch < '0' || ch > '9') {
			if(ch == EOF) {
				state = false;
				val = 0;
				return *this;
			}
			if(ch == '-') {
				f = -1;
			}
			ch = cin.rdbuf()->sbumpc();
		}
		while(ch >= '0' && ch <= '9') {
			x = x * 10 + ch - '0';
			ch = cin.rdbuf()->sbumpc();
		}
		val = x * f;
		return *this;
	}
	explicit operator bool() const noexcept {
		return state;
	}
private:
	bool state;
};
Qread qread;
```

自定义哈希

```
struct MyHash {
	size_t operator()(ll x) const noexcept {
		x ^= (x >> 21);
		x ^= (x << 37);
		x ^= (x >> 4);
		x *= 0x27d4eb2f165667c5;
		x ^= (x >> 28);
		x *= 0x165667b19e3779f9;
		x ^= (x >> 31);
		return x ^ c;
	}
private:
	static inline size_t c = (size_t)chrono::steady_clock::now().time_since_epoch().count();
};
```